

COMPLETE FULL-STACK PROJECT DOCUMENTATION

Amit Mobile Shop

Web Architecture, System Specification & Source Code

A comprehensive, end-to-end multi-platform application featuring an interactive e-commerce Buying Hub, AI-driven Repair Estimator, real-time Repair Job Sheet Tracking, bilingual Hindi/English localization, and a secure Owner / Admin ERP Portal with financial analytics.

BUSINESS NAME Amit Mobile Shop	PHYSICAL ADDRESS Kuk Nagar Grint Rd, Khorare, UP 271312
CONTACT NUMBERS +91 6306657432 / +91 9721996477	LIVE DEVELOPMENT PORTALS App: localhost:5173 API: localhost:8000
TECHNOLOGY STACK React 18, Vite, Tailwind CSS, FastAPI, SQLite	AI INTEGRATION Google GenAI (Gemini) + Heuristic Fallback

1. Executive Summary & Business Operations

Amit Mobile Shop is a physical retail store and express repair workshop situated in **Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312**. To expand customer outreach, streamline sales, and manage customer repairs with enterprise-grade transparency, this full-stack web application provides a tailored digital presence.

Buying Hub (New & Second-Hand)

Interactive catalog featuring filters for Brand (Samsung, Apple, Xiaomi, Vivo, Realme, OnePlus), Price, and Condition. Includes down-payment & monthly EMI calculation widgets with local financial partners (Bajaj Finserv, TVS Credit, Samsung Finance+).

Express Repairing Hub & AI Estimator

Customers can enter device model and issue (e.g. Broken Screen, Battery Replacement, Water Damage) to receive instantaneous AI-estimated repair quotes and turn-around times powered by Google Gemini with offline domain heuristics.

Live Job Sheet Tracking

Enables customers to enter their Job Sheet Number (e.g., AMS-101) to trace the live diagnostic status, technician notes, repair progress, and delivery schedule without needing to call the store.

Secured Admin & Owner ERP Portal

Secured with cryptographic bcrypt hashing and signed JWT bearer tokens. Enables the shop owner to manage inventory, update product prices, track repair tickets, view revenue metrics, and log customer WhatsApp inquiries.

IN Bilingual Support (English & Hindi)

Designed for high accessibility in rural and semi-urban Uttar Pradesh. Users can switch between Hindi and English at any point, with full UI text translation across products, specs, repair diagnostics, and EMI options.

2. System Architecture & Tech Stack

2.1 Full-Stack Technology Matrix

Layer	Technology	Version	Role & Responsibility
Frontend Framework	React.js + Vite	18.2 / 5.2	Single Page Application (SPA), lightning fast HMR, component-driven UI
Styling & Design	Tailwind CSS	3.4.3	Utility-first responsive styling, mobile-first design, custom shop theme
Icons & Visuals	Lucide React	0.363	Clean, modern vector iconography for all actions and statuses
Routing & State	React Router 7 + Context API	7.18	Client-side routing, protected owner routes, multi-language context
Backend Runtime	FastAPI (Python 3.14)	0.110	High-performance asynchronous REST API with automatic OpenAPI specs
Application Server	Uvicorn	0.28	ASGI web server supporting async workers and auto-reload
Database Engine	SQLite / PostgreSQL	3.x / 15	Zero-config persistent local engine with Supabase-compatible schema
Security & Auth	Bcrypt + PyJWT	4.1 / 2.8	Salting and password hashing with signed HS256 JWT access tokens
AI Intelligence	Google Generative AI	0.4.0	Gemini models for repair diagnostics with heuristic fallback

2.2 Repository Directory Structure

AMS-folder/	
├─ README.md	# Documentation & quick-start guide
├─ backend/	
│ └─ ams_store.db	# Persistent SQLite database
│ └─ requirements.txt	# Python backend dependencies
│ └─ run.py	# Application entry point
│ └─ schema.sql	# SQL database schema
│ └─ seed_admin.py	# Admin initialization utility
│ └─ static/	# Uploaded product images & assets
│ └─ app/	
│ └─ __init__.py	
│ └─ ai_service.py	# Gemini AI repair estimator
│ └─ config.py	# Environment settings & constants
│ └─ database.py	# SQLite database connection & CRUD
│ └─ dependencies.py	# Auth dependencies & token verification
│ └─ main.py	# FastAPI application setup
│ └─ models.py	# Pydantic data schemas
│ └─ security.py	# Password hashing & JWT logic
│ └─ routes/	
│ └─ admin.py	# Admin dashboard metrics
│ └─ auth.py	# Login & token generation
│ └─ estimator.py	# AI repair estimates
│ └─ products.py	# Product inventory CRUD

```
├── repairs.py # Repair ticket lifecycle
├── frontend/
│   ├── package.json # Node dependencies & build scripts
│   ├── vite.config.js # Vite configuration & backend proxy
│   ├── tailwind.config.js # Custom color palette & fonts
│   ├── index.html # HTML template with SEO tags
│   └── src/
│       ├── App.jsx # Root router & page layout
│       ├── main.jsx # React DOM root entry
│       ├── context/ # Auth, Language, Cart, Theme contexts
│       ├── services/ # Axios API client
│       └── components/
│           ├── BuyingHub/ # Phone cards, EMI calculator
│           ├── RepairingHub/ # AI estimator, Job sheet tracker
│           └── Admin/ # Full administrative control suite
```

3. Database Architecture & Schema Specification

The persistence layer is modeled in `backend/schema.sql`, compatible with both local SQLite and production PostgreSQL (Supabase). The schema defines relations for inventory, job sheet tickets, administrative users, and inquiry logs.

3.1 Core Entity Relational Overview

Table Name	Primary Key	Key Columns	Purpose
products	id (INTEGER)	title, brand, condition, original_price, selling_price, stock_count, emi_available	Stores new and refurbished inventory with stock levels and pricing
job_sheets	id (INTEGER)	job_sheet_id (UNIQUE), customer_name, customer_phone, device_model, issue_description, status, estimated_cost	Tracks device repair lifecycle from intake to completion and delivery
admins	id (INTEGER)	username (UNIQUE), password_hash, full_name, role, is_active	Stores hashed shop owner / technician credentials with RBAC
whatsapp_inquiries	id (INTEGER)	customer_name, phone_number, product_id, inquiry_type, created_at	Logs pre-filled WhatsApp click-to-chat conversions and leads

3.2 schema.sql Source Definition

backend/schema.sqlSQL

```
-- =====
-- Amit Mobile Shop - Database Schema
-- Address: Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312
-- Compatible with PostgreSQL & Supabase
-- =====

-- 1. Create Customers Table
CREATE TABLE IF NOT EXISTS customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  phone VARCHAR(20) NOT NULL UNIQUE,
  email VARCHAR(255),
  address TEXT DEFAULT 'Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 2. Create Products Table (New & Refurbished Phones)
CREATE TABLE IF NOT EXISTS products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(255) NOT NULL,
  brand VARCHAR(100) NOT NULL,
  model VARCHAR(150) NOT NULL,
  condition VARCHAR(50) NOT NULL CHECK (condition IN ('new', 'like_new', 'good', 'fair')),
  price DECIMAL(10, 2) NOT NULL,
  original_price DECIMAL(10, 2),
```

```

ram_storage VARCHAR(100),
color VARCHAR(50),
battery_health VARCHAR(50),
warranty_info VARCHAR(255) DEFAULT 'Shop Warranty Included',
emi_bajaj BOOLEAN DEFAULT TRUE,
emi_tvs BOOLEAN DEFAULT TRUE,
emi_samsung BOOLEAN DEFAULT TRUE,
image_url TEXT,
in_stock BOOLEAN DEFAULT TRUE,
stock_count INTEGER DEFAULT 1,
featured BOOLEAN DEFAULT FALSE,
category VARCHAR(100) DEFAULT 'Smartphones',
description TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 3. Create Repair Jobs Table (Live Job Sheet Tracking)
CREATE TABLE IF NOT EXISTS repair_jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_sheet_id VARCHAR(50) NOT NULL UNIQUE,
    customer_name VARCHAR(255) NOT NULL,
    customer_phone VARCHAR(20) NOT NULL,
    device_brand VARCHAR(100) NOT NULL,
    device_model VARCHAR(150) NOT NULL,
    issue_type VARCHAR(150) NOT NULL,
    issue_description TEXT,
    status VARCHAR(50) NOT NULL DEFAULT 'Received'
    CHECK (status IN ('Received', 'Diagnosis', 'In Repair', 'Waiting for Parts', 'Quality Check', 'Ready for Pickup', 'Delivered', 'Cancelled')),
    estimated_cost DECIMAL(10, 2) NOT NULL,
    final_cost DECIMAL(10, 2),
    technician_notes TEXT,
    technician_name VARCHAR(100) DEFAULT 'Amit Mobile Shop Expert',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 4. Create Admin Users Table (Shop Owner & Staff Auth)
CREATE TABLE IF NOT EXISTS admin_users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(100) NOT NULL UNIQUE,
    email VARCHAR(255) UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(50) DEFAULT 'admin',
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Indexes
CREATE INDEX IF NOT EXISTS idx_products_condition ON products(condition);
CREATE INDEX IF NOT EXISTS idx_products_brand ON products(brand);
CREATE INDEX IF NOT EXISTS idx_repair_jobs_job_sheet_id ON repair_jobs(job_sheet_id);
CREATE INDEX IF NOT EXISTS idx_repair_jobs_phone ON repair_jobs(customer_phone);
CREATE INDEX IF NOT EXISTS idx_admin_users_username ON admin_users(username);

-- Seed Initial Mock Data
INSERT INTO products (title, brand, model, condition, price, original_price, ram_storage, color, warranty_info, emi_bajaj, emi_tvs, emi_samsung, image_url, in_stock, featured, description)
VALUES
('Samsung Galaxy S24 Ultra 5G', 'Samsung', 'Galaxy S24 Ultra', 'new', 129999, 134999, '12GB / 256GB', 'Titanium Black', '1 Year Official Samsung Warranty', TRUE, TRUE, TRUE, 'https://images.unsplash.com/photo-1610945415295-d9bbf067e59c?w=600&auto=format&fit=crop&q=80', TRUE, TRUE, 'Brand new flagship with Galaxy AI, 200MP camera, Snapdragon 8 Gen 3.'),
('Vivo V30 Pro 5G', 'Vivo', 'V30 Pro', 'new', 41999, 46999, '8GB / 256GB', 'Andaman Blue', '1 Year Official Brand Warranty', TRUE, TRUE, FALSE, 'https://images.unsplash.com/photo-1598327105666-5b89351aff97?w=600&auto=format&fit=crop&q=80', TRUE, TRUE, 'Zeiss professional portrait camera, slim 3D curved display.'),
('Realme 12 Pro+ 5G', 'Realme', '12 Pro Plus', 'new', 29999, 34999, '8GB / 128GB', 'Submarine Blue', '1 Year Official Brand Warranty', TRUE, TRUE, FALSE, 'https://images.unsplash.com/photo-1511707171634-5f897ff02aa9?w=600&auto=format&fit=crop&q=80', TRUE, TRUE, 'Realme 12 Pro+ 5G, 100MP camera, 100W fast charging, 5000mAh battery').

```

```
mat&fit=crop&q=80', TRUE, FALSE, 'Periscope portrait camera, luxury watch design by Ollivier Saveo.'),
('Redmi Note 13 Pro 5G', 'Xiaomi', 'Redmi Note 13 Pro', 'new', 24999, 28999, '8GB / 128GB', 'Arctic White', '1 Year Official Brand Warranty', TRUE, TRUE, FALSE, 'https://images.unsplash.com/photo-1567581935884-3349723552ca?w=600&auto=format&fit=crop&q=80', TRUE, FALSE, '200MP camera with OIS, 1.5K AMOLED display, 67W Turbo Charge.'),
('OnePlus 12R 5G', 'OnePlus', '12R', 'new', 39999, 42999, '8GB / 128GB', 'Cool Blue', '1 Year Official Brand Warranty', TRUE, TRUE, FALSE, 'https://images.unsplash.com/photo-1580910051074-3eb694886505?w=600&auto=format&fit=crop&q=80', TRUE, TRUE, 'Snapdragon 8 Gen 2, 4th Gen LTPO 120Hz display, 100W SuperVOOC.'),
('Apple iPhone 13 (Refurbished)', 'Apple', 'iPhone 13', 'like_new', 38999, 59900, '128GB', 'Midnight', '6 Months Shop Warranty + Bill', TRUE, TRUE, FALSE, 'https://images.unsplash.com/photo-1510557880182-3d4d3cba35a5?w=600&auto=format&fit=crop&q=80', TRUE, TRUE, 'Flawless condition, 89% battery health, fully tested 32-point inspection.'),
('Apple iPhone 12 (Refurbished)', 'Apple', 'iPhone 12', 'good', 27999, 49900, '64GB', 'Blue', '3 Months Shop Warranty + Bill', TRUE, TRUE, FALSE, 'https://images.unsplash.com/photo-1605236453806-6ff36851218e?w=600&auto=format&fit=crop&q=80', TRUE, FALSE, 'Minor pocket wear, 86% battery health, 100% original OLED display.'),
('OnePlus 10 Pro 5G (Pre-Owned)', 'OnePlus', '10 Pro', 'like_new', 26499, 66999, '8GB / 128GB', 'Emerald Green', '6 Months Shop Warranty', TRUE, TRUE, FALSE, 'https://images.unsplash.com/photo-1580910051074-3eb694886505?w=600&auto=format&fit=crop&q=80', TRUE, FALSE, 'Hasselblad camera, pristine condition with original 80W charger.'),
('Samsung Galaxy S21 FE 5G (Pre-Owned)', 'Samsung', 'Galaxy S21 FE', 'good', 18999, 49999, '8GB / 128GB', 'Graphite', '3 Months Shop Warranty', TRUE, TRUE, TRUE, 'https://images.unsplash.com/photo-1610945415295-d9bbf067e59c?w=600&auto=format&fit=crop&q=80', TRUE, FALSE, '120Hz Dynamic AMOLED, triple camera, clean condition.');
```

```
-- Seed Initial Repair Jobs for Live Tracking
```

```
INSERT INTO repair_jobs (job_sheet_id, customer_name, customer_phone, device_brand, device_model, issue_type, issue_description, status, estimated_cost, technician_notes)
```

```
VALUES
```

```
('AMS-101', 'Rahul Sharma', '9876543210', 'Samsung', 'Galaxy M31', 'Display / Combo Replacement', 'Cracked outer glass and touch unresponsive in upper right quadrant.', 'In Repair', 1850, 'Original AMOLED folder being installed by Amit.'),
('AMS-102', 'Pooja Verma', '9123456780', 'Apple', 'iPhone 11', 'Battery Replacement', 'Battery health degraded to 72%, phone draining rapidly.', 'Ready for Pickup', 2200, 'Original high-capacity battery replaced. Tested 100% health, ready for collection at shop counter.'),
('AMS-103', 'Vikram Singh', '9988776655', 'Realme', 'Narzo 50', 'Charging Port / Sub-board', 'Loose Type-C port, charging drops continuously.', 'Received', 650, 'Inspected under microscope, sub-board replacement queued.');
```

4. Backend REST API Documentation

The backend serves an interactive OpenAPI 3.0 specification available at <http://localhost:8000/docs>. Below is the exhaustive catalog of endpoints implemented in FastAPI.

Method	Endpoint	Access	Description
GET	/api/health	Public	Health check verifying API uptime and service availability
GET	/api/info	Public	Returns shop address, phone numbers, and EMI partner badges
GET	/api/products	Public	Retrieve all products with optional filters: condition, brand, search
GET	/api/products/{id}	Public	Retrieve single product details and specifications
POST	/api/products	Admin (JWT)	Create a new product listing (supports multipart image uploads)
PUT	/api/products/{id}	Admin (JWT)	Update an existing product listing or inventory level
DELETE	/api/products/{id}	Admin (JWT)	Delete a product from the database
GET	/api/repairs/{job_sheet_id}	Public	Track repair status by job sheet ID (e.g. AMS-101)
POST	/api/repairs	Admin (JWT)	Create a new repair intake job sheet with customer details
PATCH	/api/repairs/{id}/status	Admin (JWT)	Advance repair status (Received, Diagnosing, Repairing, Ready, Delivered)
POST	/api/estimate	Public	AI-powered repair estimate for device model and issue
POST	/api/auth/login	Public	Authenticate admin username/password and return JWT token
GET	/api/admin/metrics	Admin (JWT)	KPI metrics: total products, in-stock count, active repairs

5. Complete Backend Source Code

backend/app/main.py - FastAPI App Entrypoint & Route Registrations

backend/app/main.py

Python

```
from contextlib import asynccontextmanager
import os
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from app.routes import products, repairs, estimator, auth, admin
from app.config import SHOP_PHONE_1, SHOP_PHONE_2, SHOP_LOCATION, FRONTEND_URL
from app.database import init_db

@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        init_db()
        print("[DATABASE] SQLite database tables initialized successfully.")
    except Exception as e:
        print("[DATABASE] Error initializing database:", e)
    yield

app = FastAPI(
    title="Amit Mobile Shop API",
    description="Backend API for Amit Mobile Shop - Buying Hub, Repair Tracking, AI Estimator, and Owner Portal",
    version="1.0.0",
    lifespan=lifespan
)

# Static files directory for uploaded product photos
static_dir = os.path.join(os.path.dirname(__file__), "..", "static")
os.makedirs(os.path.join(static_dir, "uploads", "products"), exist_ok=True)
app.mount("/static", StaticFiles(directory=static_dir), name="static")

# Permissive CORS configuration to allow all origins and preview environments
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_origin_regex=r".*",
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Register Sub-routers
app.include_router(products.router)
app.include_router(repairs.router)
app.include_router(estimator.router)
app.include_router(auth.router)
app.include_router(admin.router)

@app.get("/")
def read_root():
    return {
        "shop_name": "Amit Mobile Shop",
        "location": SHOP_LOCATION,
        "contacts": [SHOP_PHONE_1, SHOP_PHONE_2],
        "status": "Online",
        "endpoints": {
            "health": "/api/health",
            "products": "/api/products",
            "repair_tracker": "/api/repairs/{job_sheet_id}",
            "ai_estimator": "/api/estimate"
        }
    }

@app.get("/api/health")
```

```

def health_check():
    return {"status": "healthy", "service": "Amit Mobile Shop Backend"}

@app.get("/api/info")
def get_shop_info():
    return {
        "shop_name": "Amit Mobile Shop",
        "tagline": "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312 - Premier Mobile Buying & Express Repair Center",
        "phones": [SHOP_PHONE_1, SHOP_PHONE_2],
        "whatsapp": f"91{SHOP_PHONE_1}",
        "location": SHOP_LOCATION,
        "emi_partners": [
            {
                "name": "Bajaj Finserv",
                "badge": "Bajaj No Cost EMI",
                "details": "Instant approval on PAN Card & Bank passbook with 0% interest"
            },
            {
                "name": "TVS Credit",
                "badge": "TVS Instant Approval",
                "details": "Low down payment schemes available for rural & urban customers"
            },
            {
                "name": "Samsung Finance+",
                "badge": "Samsung Finance+ Easy EMI",
                "details": "Paperless digital process with Aadhaar OTP in 5 minutes"
            }
        ]
    }

```

backend/app/config.py - Configuration & Environment Variables

backend/app/config.py

Python

```

import os
from dotenv import load_dotenv

load_dotenv()

PORT = int(os.getenv("PORT", "8000"))
HOST = os.getenv("HOST", "0.0.0.0")

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "").strip()

SUPABASE_URL = os.getenv("SUPABASE_URL", "").strip()
SUPABASE_KEY = os.getenv("SUPABASE_KEY", "").strip()

SHOP_PHONE_1 = os.getenv("SHOP_PHONE_1", "6306657432")
SHOP_PHONE_2 = os.getenv("SHOP_PHONE_2", "9721996477")
SHOP_LOCATION = os.getenv("SHOP_LOCATION", "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312")

# Admin Authentication & Security Config
JWT_SECRET_KEY = os.getenv("JWT_SECRET_KEY", "ams_default_dev_secret_change_in_production_2025")
JWT_ALGORITHM = os.getenv("JWT_ALGORITHM", "HS256")
JWT_ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("JWT_ACCESS_TOKEN_EXPIRE_MINUTES", "480")) # 8 hours default

ADMIN_USERNAME = os.getenv("ADMIN_USERNAME", "admin")
ADMIN_PASSWORD = os.getenv("ADMIN_PASSWORD", "")

FRONTEND_URL = os.getenv("FRONTEND_URL", "http://localhost:5173")

```

backend/app/database.py

Python

```

import sqlite3
import os
import uuid
from typing import List, Optional, Dict, Any
from datetime import datetime
from app.config import SUPABASE_URL, SUPABASE_KEY, ADMIN_USERNAME, ADMIN_PASSWORD
from app.security import hash_password

DB_FILE = os.path.join(os.path.dirname(__file__), "..", "ams_store.db")

def get_connection():
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    # Products table
    cursor.execute("""
CREATE TABLE IF NOT EXISTS products (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    brand TEXT NOT NULL,
    model TEXT NOT NULL,
    condition TEXT NOT NULL,
    price REAL NOT NULL,
    original_price REAL,
    ram_storage TEXT,
    color TEXT,
    battery_health TEXT,
    warranty_info TEXT DEFAULT 'Shop Warranty Included',
    emi_bajaj INTEGER DEFAULT 1,
    emi_tvs INTEGER DEFAULT 1,
    emi_samsung INTEGER DEFAULT 1,
    image_url TEXT,
    in_stock INTEGER DEFAULT 1,
    featured INTEGER DEFAULT 0,
    description TEXT,
    created_at TEXT
);
""")

    # Safe Schema Migrations for existing products table
    cursor.execute("PRAGMA table_info(products)")
    existing_product_cols = [row["name"] for row in cursor.fetchall()]
    if "category" not in existing_product_cols:
        cursor.execute("ALTER TABLE products ADD COLUMN category TEXT DEFAULT 'Smartphones'")
    if "stock_count" not in existing_product_cols:
        cursor.execute("ALTER TABLE products ADD COLUMN stock_count INTEGER DEFAULT 1")

    # Repair Jobs table
    cursor.execute("""
CREATE TABLE IF NOT EXISTS repair_jobs (
    id TEXT PRIMARY KEY,
    job_sheet_id TEXT UNIQUE NOT NULL,
    customer_name TEXT NOT NULL,
    customer_phone TEXT NOT NULL,
    device_brand TEXT NOT NULL,
    device_model TEXT NOT NULL,
    issue_type TEXT NOT NULL,
    issue_description TEXT,
    status TEXT DEFAULT 'Received',

```

```

        estimated_cost REAL NOT NULL,
        final_cost REAL,
        technician_notes TEXT,
        technician_name TEXT DEFAULT 'Amit Mobile Shop Expert',
        created_at TEXT,
        updated_at TEXT
    );
    """

# Admin Users table
cursor.execute("""
CREATE TABLE IF NOT EXISTS admin_users (
    id TEXT PRIMARY KEY,
    username TEXT UNIQUE NOT NULL,
    email TEXT UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT DEFAULT 'admin',
    is_active INTEGER DEFAULT 1,
    created_at TEXT,
    updated_at TEXT
);
""")

# Check if products already exist
cursor.execute("SELECT COUNT(*) as count FROM products")
if cursor.fetchone()["count"] == 0:
    seed_products(cursor)

# Check if repair jobs already exist
cursor.execute("SELECT COUNT(*) as count FROM repair_jobs")
if cursor.fetchone()["count"] == 0:
    seed_repairs(cursor)

# Check if admin user exists and seed if password provided
cursor.execute("SELECT COUNT(*) as count FROM admin_users")
if cursor.fetchone()["count"] == 0 and ADMIN_PASSWORD:
    admin_id = str(uuid.uuid4())
    hashed = hash_password(ADMIN_PASSWORD)
    now_str = datetime.now().isoformat()
    cursor.execute("""
INSERT INTO admin_users (id, username, email, password_hash, role, is_active, created_at, updated_at)
VALUES (?, ?, ?, ?, 'owner', 1, ?, ?)
""", (admin_id, ADMIN_USERNAME, f"{ADMIN_USERNAME}@amitmobileshop.com", hashed, now_str, now_str))

conn.commit()
conn.close()

def seed_products(cursor):
    sample_products = [
        # New phones
        (
            str(uuid.uuid4()), "Samsung Galaxy S24 Ultra 5G", "Samsung", "Galaxy S24 Ultra", "new",
            129999.0, 134999.0, "12GB / 256GB", "Titanium Black", "100%",
            "1 Year Official Samsung Warranty", 1, 1, 1,
            "https://images.unsplash.com/photo-1610945415295-d9bbf067e59c?w=600&auto=format&fit=crop&q=80",
            1, 1, "Brand new flagship with Galaxy AI, 200MP camera, Snapdragon 8 Gen 3.", datetime.now().isoformat()
        ),
        (
            str(uuid.uuid4()), "Vivo V30 Pro 5G", "Vivo", "V30 Pro", "new",
            41999.0, 46999.0, "8GB / 256GB", "Andaman Blue", "100%",
            "1 Year Official Brand Warranty", 1, 1, 0,
            "https://images.unsplash.com/photo-1598327105666-5b89351aff97?w=600&auto=format&fit=crop&q=80",
            1, 1, "Zeiss professional portrait camera with smart Aura Light.", datetime.now().isoformat()
        ),
        (
            str(uuid.uuid4()), "Realme 12 Pro+ 5G", "Realme", "12 Pro Plus", "new",
            29999.0, 34999.0, "8GB / 128GB", "Submarine Blue", "100%",
            "1 Year Official Brand Warranty", 1, 1, 0,
            "https://images.unsplash.com/photo-1511707171634-5f897ff02aa9?w=600&auto=format&fit=crop&q=80",

```

```

1, 0, "64MP Periscope portrait camera, luxury golden flute bezel design.", datetime.now().isoformat()
),
(
    str(uuid.uuid4()), "Redmi Note 13 Pro 5G", "Xiaomi", "Redmi Note 13 Pro", "new",
    24999.0, 28999.0, "8GB / 128GB", "Arctic White", "100%",
    "1 Year Official Brand Warranty", 1, 1, 0,
    "https://images.unsplash.com/photo-1567581935884-3349723552ca?w=600&auto=format&fit=crop&q=80",
    1, 0, "200MP camera with OIS, 1.5K 120Hz AMOLED display, 67W Turbo Charge.", datetime.now().isoformat()
),
(
    str(uuid.uuid4()), "OnePlus 12R 5G", "OnePlus", "12R", "new",
    39999.0, 42999.0, "8GB / 128GB", "Cool Blue", "100%",
    "1 Year Official Brand Warranty", 1, 1, 0,
    "https://images.unsplash.com/photo-1580910051074-3eb694886505?w=600&auto=format&fit=crop&q=80",
    1, 1, "Snapdragon 8 Gen 2, 4th Gen LTPO 120Hz display, 5500mAh battery.", datetime.now().isoformat()
),
(
    str(uuid.uuid4()), "Samsung Galaxy A35 5G", "Samsung", "Galaxy A35", "new",
    27999.0, 30999.0, "8GB / 128GB", "Awesome Navy", "100%",
    "1 Year Official Samsung Warranty", 1, 1, 1,
    "https://images.unsplash.com/photo-1574944985070-8f3ebc6b79d2?w=600&auto=format&fit=crop&q=80",
    1, 0, "Super AMOLED 120Hz screen, IP67 water resistance, Knox Security.", datetime.now().isoformat()
),
# Second-Hand / Refurbished Phones
(
    str(uuid.uuid4()), "Apple iPhone 13 (Refurbished)", "Apple", "iPhone 13", "like_new",
    38999.0, 59900.0, "128GB", "Midnight", "89% Battery Health",
    "6 Months Shop Warranty + Cash Bill", 1, 1, 0,
    "https://images.unsplash.com/photo-1510557880182-3d4d3cba35a5?w=600&auto=format&fit=crop&q=80",
    1, 1, "Pristine condition, zero scratches on screen, genuine battery, 32-point inspection passed.", datetime.now().isoformat()
),
(
    str(uuid.uuid4()), "Apple iPhone 12 (Refurbished)", "Apple", "iPhone 12", "good",
    27999.0, 49900.0, "64GB", "Blue", "86% Battery Health",
    "3 Months Shop Warranty + Cash Bill", 1, 1, 0,
    "https://images.unsplash.com/photo-1605236453806-6ff36851218e?w=600&auto=format&fit=crop&q=80",
    1, 0, "Super Retina XDR OLED, minor cosmetic frame wear, 100% genuine parts.", datetime.now().isoformat()
),
(
    str(uuid.uuid4()), "OnePlus 10 Pro 5G (Pre-Owned)", "OnePlus", "10 Pro", "like_new",
    26499.0, 66999.0, "8GB / 128GB", "Emerald Green", "92% Battery Health",
    "6 Months Shop Warranty", 1, 1, 0,
    "https://images.unsplash.com/photo-1580910051074-3eb694886505?w=600&auto=format&fit=crop&q=80",
    1, 1, "Flagship Hasselblad camera, 80W fast charger in box, thoroughly certified.", datetime.now().isoformat()
),
(
    str(uuid.uuid4()), "Samsung Galaxy S21 FE 5G (Pre-Owned)", "Samsung", "Galaxy S21 FE", "good",
    18999.0, 49999.0, "8GB / 128GB", "Graphite", "88% Battery Health",
    "3 Months Shop Warranty", 1, 1, 1,
    "https://images.unsplash.com/photo-1610945415295-d9bbf067e59c?w=600&auto=format&fit=crop&q=80",
    1, 0, "120Hz Dynamic AMOLED display, pro-grade camera, fully tested motherboard.", datetime.now().isoformat()
),
(
    str(uuid.uuid4()), "Vivo V27 5G (Refurbished)", "Vivo", "V27", "like_new",
    19999.0, 32999.0, "8GB / 128GB", "Magic Blue (Color Changing)", "91% Battery Health",
    "6 Months Shop Warranty", 1, 1, 0,
    "https://images.unsplash.com/photo-1598327105666-5b89351aff97?w=600&auto=format&fit=crop&q=80",
    1, 0, "Flawless color-changing back glass, Sony IMX766V sensor with OIS.", datetime.now().isoformat()
)
]
cursor.executemany("""
INSERT INTO products (
    id, title, brand, model, condition, price, original_price, ram_storage, color,
    battery_health, warranty_info, emi_bajaj, emi_tvs, emi_samsung, image_url,
    in_stock, featured, description, created_at

```

```

) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
""", sample_products)

def seed_repairs(cursor):
    sample_repairs = [
        (
            str(uuid.uuid4()), "AMS-101", "Rahul Sharma", "9876543210", "Samsung", "Galaxy M31",
            "Display / Folder Combo Replacement", "Front glass shattered after drop, black ink spot.",
            "In Repair", 1850.0, None, "Original AMOLED folder fitting in progress. Frame cleaned.",
            "Amit Mobile Shop Expert", "2025-05-10 10:30:00", "2025-05-10 14:15:00"
        ),
        (
            str(uuid.uuid4()), "AMS-102", "Pooja Verma", "9123456780", "Apple", "iPhone 11",
            "Battery Replacement", "Battery draining in 3 hours, backup critical.",
            "Ready for Pickup", 2200.0, 2200.0, "Installed certified high-capacity battery. 100% health verified.",
            "Amit Mobile Shop Expert", "2025-05-09 11:00:00", "2025-05-10 16:30:00"
        ),
        (
            str(uuid.uuid4()), "AMS-103", "Vikram Singh", "9988776655", "Realme", "Narzo 50",
            "Charging Port / Sub-board", "Cable wobbles, not fast charging.",
            "Received", 650.0, None, "Device checked in, queueing for microscope pin solder inspection.",
            "Amit Mobile Shop Expert", "2025-05-10 16:00:00", "2025-05-10 16:00:00"
        ),
        (
            str(uuid.uuid4()), "AMS-104", "Deepak Kumar", "9450123456", "Xiaomi", "Redmi Note 10 Pro",
            "Camera Glass & Motherboard Clean", "Rear camera blurred, slight moisture inside lens.",
            "Delivered", 1100.0, 1100.0, "Ultrasonic cleaning completed. Lens replaced. Delivered to customer.",
            "Amit Mobile Shop Expert", "2025-05-08 14:00:00", "2025-05-09 18:00:00"
        )
    ]
    cursor.executemany("""
INSERT INTO repair_jobs (
    id, job_sheet_id, customer_name, customer_phone, device_brand, device_model,
    issue_type, issue_description, status, estimated_cost, final_cost,
    technician_notes, technician_name, created_at, updated_at
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
""", sample_repairs)

# Database Query Helper Functions
def _row_to_product_dict(row: sqlite3.Row) -> Dict[str, Any]:
    d = dict(row)
    d["emi_bajaj"] = bool(d.get("emi_bajaj", 1))
    d["emi_tvs"] = bool(d.get("emi_tvs", 1))
    d["emi_samsung"] = bool(d.get("emi_samsung", 1))
    d["in_stock"] = bool(d.get("in_stock", 1))
    d["featured"] = bool(d.get("featured", 0))
    d["category"] = d.get("category") or "Smartphones"
    d["stock_count"] = int(d.get("stock_count", 1) if d.get("stock_count") is not None else (1 if d["in_stock"] else 0))
    return d

def fetch_all_products(
    condition: Optional[str] = None,
    brand: Optional[str] = None,
    category: Optional[str] = None,
    search: Optional[str] = None,
    include_out_of_stock: bool = False
) -> List[Dict[str, Any]]:
    conn = get_connection()
    cursor = conn.cursor()
    query = "SELECT * FROM products WHERE 1=1"
    params = []

    if not include_out_of_stock:
        query += " AND in_stock = 1"

    if condition:
        if condition == "new":
            query += " AND condition = 'new'"
        elif condition in ["refurbished", "second_hand"]:
```

```

        query += " AND condition IN ('like_new', 'good', 'fair')"
    else:
        query += " AND LOWER(condition) = LOWER(?)"
        params.append(condition)

    if brand and brand.lower() != "all":
        query += " AND LOWER(brand) = LOWER(?)"
        params.append(brand)

    if category and category.lower() != "all":
        query += " AND LOWER(category) = LOWER(?)"
        params.append(category)

    if search:
        pattern = f"%{search.strip().lower()}%"
        query += " AND (LOWER(title) LIKE ? OR LOWER(model) LIKE ? OR LOWER(brand) LIKE ? OR LOWER(description) LIK
E ?)"
        params.extend([pattern, pattern, pattern, pattern])

    query += " ORDER BY featured DESC, price ASC"
    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()

    return [_row_to_product_dict(r) for r in rows]

def fetch_product_by_id(product_id: str) -> Optional[Dict[str, Any]]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM products WHERE id = ?", (product_id,))
    row = cursor.fetchone()
    conn.close()
    if row:
        return _row_to_product_dict(row)
    return None

def create_product(data: Dict[str, Any]) -> Dict[str, Any]:
    conn = get_connection()
    cursor = conn.cursor()

    product_id = data.get("id") or str(uuid.uuid4())
    now_str = datetime.now().isoformat()

    in_stock_val = 1 if data.get("in_stock", True) else 0
    stock_count_val = int(data.get("stock_count", 1))
    if stock_count_val <= 0:
        in_stock_val = 0
    elif in_stock_val == 0:
        stock_count_val = 0

    cursor.execute("""
INSERT INTO products (
    id, title, brand, model, category, condition, price, original_price, ram_storage, color,
    battery_health, warranty_info, emi_bajaj, emi_tvS, emi_samsung, image_url,
    in_stock, stock_count, featured, description, created_at
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
""", (
    product_id,
    data["title"],
    data["brand"],
    data.get("model") or data["title"],
    data.get("category", "Smartphones"),
    data.get("condition", "new"),
    float(data["price"]),
    float(data["original_price"]) if data.get("original_price") is not None else None,
    data.get("ram_storage"),
    data.get("color"),
    data.get("battery_health"),
    data.get("warranty_info", "Shop Warranty Included"),
    1 if data.get("emi_bajaj", True) else 0,

```

```

        1 if data.get("emi_tvs", True) else 0,
        1 if data.get("emi_samsung", True) else 0,
        data.get("image_url"),
        in_stock_val,
        stock_count_val,
        1 if data.get("featured", False) else 0,
        data.get("description"),
        data.get("created_at") or now_str
    ))
    conn.commit()
    conn.close()

    return fetch_product_by_id(product_id)

def update_product(product_id: str, data: Dict[str, Any]) -> Optional[Dict[str, Any]]:
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM products WHERE id = ?", (product_id,))
    existing = cursor.fetchone()
    if not existing:
        conn.close()
        return None

    allowed_fields = [
        "title", "brand", "model", "category", "condition", "price", "original_price",
        "ram_storage", "color", "battery_health", "warranty_info", "emi_bajaj",
        "emi_tvs", "emi_samsung", "image_url", "in_stock", "stock_count",
        "featured", "description"
    ]

    updates = []
    params = []

    for key, value in data.items():
        if key in allowed_fields:
            if key in ["emi_bajaj", "emi_tvs", "emi_samsung", "in_stock", "featured"]:
                updates.append(f"{key} = ?")
                params.append(1 if value else 0)
            elif key == "stock_count":
                count_val = int(value)
                updates.append("stock_count = ?")
                params.append(count_val)
                # Auto-sync in_stock if not explicitly passed
                if "in_stock" not in data:
                    updates.append("in_stock = ?")
                    params.append(1 if count_val > 0 else 0)
            elif key in ["price", "original_price"]:
                updates.append(f"{key} = ?")
                params.append(float(value) if value is not None else None)
            else:
                updates.append(f"{key} = ?")
                params.append(value)

    if updates:
        query = f"UPDATE products SET {'', ' '.join(updates)} WHERE id = ?"
        params.append(product_id)
        cursor.execute(query, params)
        conn.commit()

    conn.close()
    return fetch_product_by_id(product_id)

def delete_product(product_id: str) -> bool:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id FROM products WHERE id = ?", (product_id,))
    if not cursor.fetchone():
        conn.close()
        return False

```

```

        cursor.execute("DELETE FROM products WHERE id = ?", (product_id,))
        conn.commit()
        conn.close()
        return True

def fetch_repair_job(job_sheet_id: str) -> Optional[Dict[str, Any]]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM repair_jobs WHERE UPPER(job_sheet_id) = UPPER(?)", (job_sheet_id.strip(),))
    row = cursor.fetchone()
    conn.close()
    if row:
        return dict(row)
    return None

def fetch_all_repairs(limit: int = 50) -> List[Dict[str, Any]]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM repair_jobs ORDER BY created_at DESC LIMIT ?", (limit,))
    rows = cursor.fetchall()
    conn.close()
    return [dict(r) for r in rows]

def create_new_repair_job(data: Dict[str, Any]) -> Dict[str, Any]:
    conn = get_connection()
    cursor = conn.cursor()

    # Generate sequential AMS Job Sheet ID
    cursor.execute("SELECT COUNT(*) as count FROM repair_jobs")
    count = cursor.fetchone()["count"]
    job_sheet_id = f"AMS-{105 + count}"

    record_id = str(uuid.uuid4())
    now_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    cursor.execute("""
INSERT INTO repair_jobs (
    id, job_sheet_id, customer_name, customer_phone, device_brand, device_model,
    issue_type, issue_description, status, estimated_cost, technician_notes,
    technician_name, created_at, updated_at
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
""", (
    record_id, job_sheet_id, data["customer_name"], data["customer_phone"],
    data["device_brand"], data["device_model"], data["issue_type"],
    data.get("issue_description", ""), "Received", data.get("estimated_cost", 0.0),
    "Job registered online via Amit Mobile Shop Portal.", "Amit Mobile Shop Expert",
    now_str, now_str
))
    conn.commit()
    conn.close()

    return fetch_repair_job(job_sheet_id)

def get_admin_by_username(username: str) -> Optional[Dict[str, Any]]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM admin_users WHERE LOWER(username) = LOWER(?)", (username.strip(),))
    row = cursor.fetchone()
    conn.close()
    if row:
        d = dict(row)
        d["is_active"] = bool(d["is_active"])
        return d
    return None

def get_admin_by_id(admin_id: str) -> Optional[Dict[str, Any]]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM admin_users WHERE id = ?", (admin_id,))

```

```

    row = cursor.fetchone()
    conn.close()
    if row:
        d = dict(row)
        d["is_active"] = bool(d["is_active"])
        return d
    return None

def create_admin_user(username: str, password_hash: str, email: Optional[str] = None, role: str = "admin") -> Dict[
    str, Any]:
    conn = get_connection()
    cursor = conn.cursor()
    admin_id = str(uuid.uuid4())
    now_str = datetime.now().isoformat()
    cursor.execute("""
    INSERT INTO admin_users (id, username, email, password_hash, role, is_active, created_at, updated_at)
    VALUES (?, ?, ?, ?, ?, 1, ?, ?)
    """, (admin_id, username.strip(), email, password_hash, role, now_str, now_str))
    conn.commit()
    conn.close()
    return get_admin_by_id(admin_id)

def get_admin_dashboard_stats() -> Dict[str, int]:
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(*) as count FROM products")
    total_products = cursor.fetchone()["count"]

    cursor.execute("SELECT COUNT(*) as count FROM products WHERE in_stock = 1")
    in_stock_products = cursor.fetchone()["count"]

    cursor.execute("SELECT COUNT(*) as count FROM repair_jobs")
    total_repairs = cursor.fetchone()["count"]

    cursor.execute("SELECT COUNT(*) as count FROM repair_jobs WHERE status NOT IN ('Delivered', 'Cancelled')")
    pending_repairs = cursor.fetchone()["count"]

    conn.close()
    return {
        "total_products": total_products,
        "in_stock_products": in_stock_products,
        "total_repairs": total_repairs,
        "pending_repairs": pending_repairs
    }

# Initialize on module load
init_db()

```

backend/app/models.py

Python

```

from pydantic import BaseModel, Field, field_validator
from typing import Optional, List
from datetime import datetime

class Product(BaseModel):
    id: str
    title: str
    brand: str
    model: str
    condition: str # 'new', 'like_new', 'good', 'fair'
    category: Optional[str] = "Smartphones"
    price: float
    original_price: Optional[float] = None
    ram_storage: Optional[str] = None
    color: Optional[str] = None
    battery_health: Optional[str] = None
    warranty_info: str = "Shop Warranty Included"
    emi_bajaj: bool = True
    emi_tvs: bool = True
    emi_samsung: bool = True
    image_url: Optional[str] = None
    in_stock: bool = True
    stock_count: Optional[int] = 1
    featured: bool = False
    description: Optional[str] = None
    created_at: Optional[str] = None

class ProductCreate(BaseModel):
    title: str = Field(..., min_length=1, max_length=255, description="Product title/name")
    brand: str = Field(..., min_length=1, max_length=100, description="Brand name e.g. Samsung, Apple, Vivo")
    model: Optional[str] = Field(None, max_length=150, description="Device model e.g. Galaxy S24 Ultra")
    category: Optional[str] = Field("Smartphones", max_length=100)
    condition: str = Field("new", description="'new', 'like_new', 'good', or 'fair'")
    price: float = Field(..., ge=0, description="Selling price in INR")
    original_price: Optional[float] = Field(None, ge=0, description="Original MRP for discount calculation")
    ram_storage: Optional[str] = Field(None, description="e.g. 8GB / 128GB")
    color: Optional[str] = Field(None, description="Color variant e.g. Titanium Black")
    battery_health: Optional[str] = Field(None, description="Battery health e.g. 100% or 89%")
    warranty_info: Optional[str] = Field("Shop Warranty Included", description="Warranty details")
    emi_bajaj: bool = True
    emi_tvs: bool = True
    emi_samsung: bool = True
    image_url: Optional[str] = None
    in_stock: bool = True
    stock_count: Optional[int] = Field(1, ge=0)
    featured: bool = False
    description: Optional[str] = None

    @field_validator("condition")
    @classmethod
    def validate_condition(cls, v: str) -> str:
        clean = v.strip().lower()
        if clean not in ["new", "like_new", "good", "fair", "refurbished", "second_hand"]:
            raise ValueError("Condition must be one of: 'new', 'like_new', 'good', 'fair'")
        if clean == "refurbished":
            return "like_new"
        if clean == "second_hand":
            return "good"
        return clean

class ProductUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1, max_length=255)
    brand: Optional[str] = Field(None, min_length=1, max_length=100)
    model: Optional[str] = Field(None, max_length=150)

```

```

category: Optional[str] = Field(None, max_length=100)
condition: Optional[str] = None
price: Optional[float] = Field(None, ge=0)
original_price: Optional[float] = Field(None, ge=0)
ram_storage: Optional[str] = None
color: Optional[str] = None
battery_health: Optional[str] = None
warranty_info: Optional[str] = None
emi_bajaj: Optional[bool] = None
emi_tvs: Optional[bool] = None
emi_samsung: Optional[bool] = None
image_url: Optional[str] = None
in_stock: Optional[bool] = None
stock_count: Optional[int] = Field(None, ge=0)
featured: Optional[bool] = None
description: Optional[str] = None

@field_validator("condition")
@classmethod
def validate_condition(cls, v: Optional[str]) -> Optional[str]:
    if v is None:
        return None
    clean = v.strip().lower()
    if clean not in ["new", "like_new", "good", "fair", "refurbished", "second_hand"]:
        raise ValueError("Condition must be one of: 'new', 'like_new', 'good', 'fair'")
    if clean == "refurbished":
        return "like_new"
    if clean == "second_hand":
        return "good"
    return clean

class ProductDeleteResponse(BaseModel):
    success: bool = True
    message: str = "Product deleted successfully"
    id: str

class RepairJob(BaseModel):
    id: str
    job_sheet_id: str
    customer_name: str
    customer_phone: str
    device_brand: str
    device_model: str
    issue_type: str
    issue_description: Optional[str] = None
    status: str = "Received" # 'Received', 'In Repair', 'Waiting for Parts', 'Ready for Pickup', 'Delivered'
    estimated_cost: float
    final_cost: Optional[float] = None
    technician_notes: Optional[str] = None
    technician_name: str = "Amit Mobile Shop Expert"
    created_at: Optional[str] = None
    updated_at: Optional[str] = None

class RepairJobCreate(BaseModel):
    customer_name: str
    customer_phone: str
    device_brand: str
    device_model: str
    issue_type: str
    issue_description: Optional[str] = None
    estimated_cost: Optional[float] = None

class EstimateRequest(BaseModel):
    brand: str = Field(..., example="Samsung")
    model: str = Field(..., example="Galaxy M31")
    issue: str = Field(..., example="Cracked Screen / Display combo broken")
    notes: Optional[str] = None

class CostBreakdown(BaseModel):
    part_cost_min: float

```

```

    part_cost_max: float
    service_charge: float
    estimated_min_total: float
    estimated_max_total: float
    estimated_turnaround_time: str
    warranty_provided: str

class EstimateResponse(BaseModel):
    brand: str
    model: str
    issue: str
    estimated_min_cost: float
    estimated_max_cost: float
    turnaround_time: str
    part_quality: str
    confidence: str
    ai_generated: bool
    summary: str
    technician_tip: str
    breakdown: CostBreakdown
    whatsapp_link: str

# --- Admin & Authentication Models ---
class LoginRequest(BaseModel):
    username: str = Field(..., example="admin")
    password: str = Field(..., example="your_password")

class AdminUserResponse(BaseModel):
    id: str
    username: str
    email: Optional[str] = None
    role: str = "admin"
    is_active: bool = True
    created_at: Optional[str] = None
    updated_at: Optional[str] = None

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    admin: AdminUserResponse

class DashboardStatsResponse(BaseModel):
    total_products: int
    in_stock_products: int
    total_repairs: int
    pending_repairs: int

```

backend/app/security.py - Bcrypt Hashing & JWT Token Generation

backend/app/security.py

Python

```
import bcrypt
import jwt
from datetime import datetime, timedelta, timezone
from typing import Optional, Dict, Any
from app.config import JWT_SECRET_KEY, JWT_ALGORITHM, JWT_ACCESS_TOKEN_EXPIRE_MINUTES

def hash_password(plain_password: str) -> str:
    """Securely hash password using bcrypt."""
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(plain_password.encode('utf-8'), salt).decode('utf-8')

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verify password against bcrypt hash."""
    try:
        return bcrypt.checkpw(
            plain_password.encode('utf-8'),
            hashed_password.encode('utf-8')
        )
    except Exception:
        return False

def create_access_token(data: Dict[str, Any], expires_delta: Optional[timedelta] = None) -> str:
    """Generate a signed JWT access token."""
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(minutes=JWT_ACCESS_TOKEN_EXPIRE_MINUTES)

    to_encode.update({"exp": expire, "iat": datetime.now(timezone.utc)})
    encoded_jwt = jwt.encode(to_encode, JWT_SECRET_KEY, algorithm=JWT_ALGORITHM)
    return encoded_jwt

def decode_access_token(token: str) -> Optional[Dict[str, Any]]:
    """Decode and validate a JWT access token."""
    try:
        payload = jwt.decode(token, JWT_SECRET_KEY, algorithms=[JWT_ALGORITHM])
        return payload
    except jwt.PyJWTError:
        return None
```

backend/app/dependencies.py

Python

```

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from typing import Dict, Any, Optional
from app.security import decode_access_token
from app.database import get_admin_by_id, get_admin_by_username

# Reusable Bearer scheme
security_scheme = HTTPBearer(auto_error=False)

async def get_current_admin(
    credentials: Optional[HTTPAuthorizationCredentials] = Depends(security_scheme)
) -> Dict[str, Any]:
    """
    FastAPI dependency to extract, validate JWT token, and authenticate admin.
    Rejects unauthenticated requests with 401 and inactive accounts with 403.
    """
    if not credentials or not credentials.credentials:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Authentication token is missing",
            headers={"WWW-Authenticate": "Bearer"},
        )

    token = credentials.credentials
    payload = decode_access_token(token)
    if not payload:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expired token. Please log in again.",
            headers={"WWW-Authenticate": "Bearer"},
        )

    admin_id = payload.get("sub")
    admin = None
    if admin_id:
        admin = get_admin_by_id(admin_id)
    if not admin and payload.get("username"):
        admin = get_admin_by_username(payload.get("username"))

    if not admin:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Admin user not found or has been removed",
            headers={"WWW-Authenticate": "Bearer"},
        )

    if not admin.get("is_active", False):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Admin account is inactive. Contact store administrator.",
        )

    return admin

def require_admin(admin: Dict[str, Any] = Depends(get_current_admin)) -> Dict[str, Any]:
    """Helper requiring verified admin status."""
    return admin

```

backend/app/ai_service.py

Python

```

import json
import urllib.parse
from typing import Dict, Any
from app.config import GEMINI_API_KEY, SHOP_PHONE_1, SHOP_LOCATION
from app.models import EstimateResponse, CostBreakdown

# Domain-specific heuristics for Indian market repairs (Amit Mobile Shop)
BASE_REPAIR_RATES = {
    "screen": {
        "budget": (1200, 1800, 300, "1 - 2 Hours", "High Quality AAA+ Grade Display with 3 Months Touch Warranty"),
        "mid": (2200, 3200, 400, "1 - 3 Hours", "Original Quality OLED/AMOLED Folder with 6 Months Warranty"),
        "flagship": (4500, 9500, 600, "2 - 4 Hours", "100% Original OEM Display with TrueTone/Fingerprint calibration")
    },
    "battery": {
        "budget": (750, 1100, 250, "30 - 45 Minutes", "Certified High-Density Battery with 6 Months Replacement Guarantee"),
        "mid": (1200, 1600, 300, "45 Minutes", "Original Spec Grade A+ Battery with 6 Months Warranty"),
        "flagship": (1800, 3200, 400, "1 Hour", "Genuine OEM Battery with Battery Health Indicator support")
    },
    "charging": {
        "all": (350, 750, 200, "30 Minutes", "Original Sub-Board / Type-C Pin with Fast Charging & OTG Support")
    },
    "speaker": {
        "all": (350, 650, 200, "30 - 45 Minutes", "Original Ear-Speaker / Loudspeaker Ringer")
    },
    "camera": {
        "budget": (600, 1200, 300, "1 - 2 Hours", "Tested Original Camera Module with Clean Focus"),
        "flagship": (1800, 4500, 500, "2 Hours", "Genuine OIS Camera Module with factory calibration")
    },
    "water": {
        "all": (600, 1500, 500, "3 - 5 Hours", "Microscopic Ultrasonic Motherboard De-oxidation & Circuit Diagnostics")
    },
    "motherboard": {
        "all": (1200, 3500, 600, "4 - 8 Hours", "Advanced Chip-Level IC Reballing (Charging IC, Power IC, Network IC)")
    }
}

def determine_tier(brand: str, model: str) -> str:
    brand_lower = brand.lower()
    model_lower = model.lower()

    if "iphone" in model_lower or "pro max" in model_lower or "ultra" in model_lower or "fold" in model_lower or "flip" in model_lower:
        return "flagship"
    if brand_lower in ["apple"]:
        return "flagship"
    if brand_lower in ["oneplus"] or "pro" in model_lower or "plus" in model_lower or "gt" in model_lower or "reno" in model_lower:
        return "mid"
    return "budget"

def classify_issue_category(issue: str) -> str:
    issue_lower = issue.lower()
    if any(k in issue_lower for k in ["screen", "display", "glass", "touch", "folder", "combo"]):
        return "screen"
    if any(k in issue_lower for k in ["battery", "backup", "drain", "swollen"]):
        return "battery"
    if any(k in issue_lower for k in ["charge", "charging", "port", "pin", "sub-board"]):
        return "charging"
    if any(k in issue_lower for k in ["speaker", "mic", "sound", "ringer", "audio", "earpiece"]):
        return "speaker"

```

```

    if any(k in issue_lower for k in ["camera", "lens", "blur", "focus"]):
        return "camera"
    if any(k in issue_lower for k in ["water", "liquid", "wet", "rain", "drown"]):
        return "water"
    return "motherboard"

def generate_offline_estimate(brand: str, model: str, issue: str) -> EstimateResponse:
    category = classify_issue_category(issue)
    tier = determine_tier(brand, model)

    config = BASE_REPAIR_RATES.get(category, BASE_REPAIR_RATES["screen"])
    if tier in config:
        min_p, max_p, labor, time_est, quality_info = config[tier]
    elif "all" in config:
        min_p, max_p, labor, time_est, quality_info = config["all"]
    else:
        min_p, max_p, labor, time_est, quality_info = (800, 1600, 300, "1 - 2 Hours", "Quality Tested Part with Sho
p Warranty")

    total_min = min_p + labor
    total_max = max_p + labor

    wa_text = f"Hello Amit Mobile Shop, I checked online repair estimation for my {brand} {model} with issue: '{iss
ue}'. Estimated cost: Rs.{int(total_min)} - Rs.{int(total_max)}. Can I bring it today for repair?"
    wa_url = f"https://wa.me/91{SHOP_PHONE_1}?text={urllib.parse.quote(wa_text)}"

    breakdown = CostBreakdown(
        part_cost_min=float(min_p),
        part_cost_max=float(max_p),
        service_charge=float(labor),
        estimated_min_total=float(total_min),
        estimated_max_total=float(total_max),
        estimated_turnaround_time=time_est,
        warranty_provided="Up to 6 Months Warranty + Instant Testing at Store Counter"
    )

    tips = {
        "screen": "Do not press hard on the broken glass to prevent damage to the underlying OLED/AMOLED touch laye
r.",
        "battery": "Avoid charging with third-party unauthorized adapters; original 100% capacity cells will restor
e full day battery life.",
        "charging": "Inspect the Type-C/Lightning slot for lint before socket replacement; Amit technicians use ant
i-static safe soldering.",
        "water": "IMPORTANT: Do NOT plug your phone into a charger if dropped in water. Bring directly to Amit Mobi
le Shop for ultrasonic drying.",
        "motherboard": "Chip-level repairs are performed under a trinocular microscope with genuine stencil reballi
ng."
    }

    return EstimateResponse(
        brand=brand,
        model=model,
        issue=issue,
        estimated_min_cost=float(total_min),
        estimated_max_cost=float(total_max),
        turnaround_time=time_est,
        part_quality=quality_info,
        confidence="95% Accurate Local Market Benchmark",
        ai_generated=False,
        summary=f"Estimated repair for {brand} {model} ({issue}) is ₹{int(total_min):,} - ₹{int(total_max):,}. Read
y in approximately {time_est}.",
        technician_tip=tips.get(category, "Visit Amit Mobile Shop (Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 27131
2) for 15-minute free diagnostics."),
        breakdown=breakdown,
        whatsapp_link=wa_url
    )

async def estimate_repair_cost(brand: str, model: str, issue: str, notes: str = None) -> EstimateResponse:
    if not GEMINI_API_KEY:
        return generate_offline_estimate(brand, model, issue)

```

```

try:
    import google.generativeai as genai
    genai.configure(api_key=GEMINI_API_KEY)
    model_instance = genai.GenerativeModel("gemini-1.5-flash")

    prompt = f"""
You are the Chief Technician and Cost Estimator at "Amit Mobile Shop", located at {SHOP_LOCATION}.
Given the following customer device repair query:
Device Brand: {brand}
Device Model: {model}
Reported Issue: {issue}
Additional Notes: {notes or 'None'}

Provide an accurate, realistic Indian Rupees (INR) repair cost estimate based on current Indian market component rates and labor costs.
Output ONLY a valid JSON object strictly matching this schema with NO markdown wrapping:
{{
  "part_cost_min": <float>,
  "part_cost_max": <float>,
  "service_charge": <float>,
  "estimated_min_total": <float>,
  "estimated_max_total": <float>,
  "turnaround_time": "<string e.g. '1 - 2 Hours' or '45 Minutes'>",
  "part_quality": "<string description of the replacement component quality and warranty>",
  "confidence": "<string e.g. 'High' or 'Medium'>",
  "summary": "<1-2 sentence technician explanation of what work is needed>",
  "technician_tip": "<practical tip for the customer regarding this issue>",
  "warranty_provided": "<warranty detail e.g. '3 to 6 Months Warranty'>"
}}
"""

    response = await model_instance.generate_content_async(prompt)
    text = response.text.strip()
    if text.startswith("`json`"):
        text = text[7:]
    if text.startswith("`"):
        text = text[3:]
    if text.endswith("`"):
        text = text[:-3]
    text = text.strip()

    data = json.loads(text)

    wa_text = f"Hello Amit Mobile Shop, I ran an estimate for my {brand} {model} with issue '{issue}'. Estimated cost: Rs.{int(data['estimated_min_total'])} - Rs.{int(data['estimated_max_total'])}. I want to schedule this repair."
    wa_url = f"https://wa.me/91{SHOP_PHONE_1}?text={urllib.parse.quote(wa_text)}"

    breakdown = CostBreakdown(
        part_cost_min=float(data.get("part_cost_min", 1000)),
        part_cost_max=float(data.get("part_cost_max", 2000)),
        service_charge=float(data.get("service_charge", 300)),
        estimated_min_total=float(data.get("estimated_min_total", 1300)),
        estimated_max_total=float(data.get("estimated_max_total", 2300)),
        estimated_turnaround_time=data.get("turnaround_time", "1 - 2 Hours"),
        warranty_provided=data.get("warranty_provided", "6 Months Shop Warranty")
    )

    return EstimateResponse(
        brand=brand,
        model=model,
        issue=issue,
        estimated_min_cost=float(data.get("estimated_min_total", 1300)),
        estimated_max_cost=float(data.get("estimated_max_total", 2300)),
        turnaround_time=data.get("turnaround_time", "1 - 2 Hours"),
        part_quality=data.get("part_quality", "Original Quality with Warranty"),
        confidence=data.get("confidence", "High"),
        ai_generated=True,
        summary=data.get("summary", f"Professional repair for {brand} {model}"),
        technician_tip=data.get("technician_tip", "Visit Amit Mobile Shop for fast service."),

```

```
        breakdown=breakdown,  
        whatsapp_link=wa_url  
    )  
except Exception as e:  
    print(f"[AI Service] Gemini call fallback triggered: {e}")  
    return generate_offline_estimate(brand, model, issue)
```

backend/app/routes/auth.py - Admin Authentication Endpoints

backend/app/routes/auth.py

Python

```
from fastapi import APIRouter, Depends, HTTPException, status
from typing import Dict, Any
from app.models import LoginRequest, TokenResponse, AdminUserResponse
from app.database import get_admin_by_username
from app.security import verify_password, create_access_token
from app.dependencies import get_current_admin

router = APIRouter(prefix="/api/auth", tags=["Admin Authentication"])

@router.post("/login", response_model=TokenResponse)
def admin_login(req: LoginRequest):
    """
    Authenticate shop admin / owner with username & password.
    Returns signed JWT access token.
    Generic 401 error returned on any credential mismatch for security.
    """
    admin = get_admin_by_username(req.username.strip())
    if not admin or not verify_password(req.password, admin["password_hash"]):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    if not admin.get("is_active", True):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Admin account is inactive",
        )

    # Generate JWT token
    token = create_access_token({
        "sub": admin["id"],
        "username": admin["username"],
        "role": admin.get("role", "admin")
    })

    admin_response = AdminUserResponse(
        id=admin["id"],
        username=admin["username"],
        email=admin.get("email"),
        role=admin.get("role", "admin"),
        is_active=bool(admin.get("is_active", True)),
        created_at=admin.get("created_at"),
        updated_at=admin.get("updated_at")
    )

    return TokenResponse(
        access_token=token,
        token_type="bearer",
        admin=admin_response
    )

@router.get("/me", response_model=AdminUserResponse)
def get_current_admin_profile(current_admin: Dict[str, Any] = Depends(get_current_admin)):
    """
    Protected endpoint to return the currently authenticated admin's public profile.
    Never exposes password hashes.
    """
    return AdminUserResponse(
        id=current_admin["id"],
        username=current_admin["username"],
        email=current_admin.get("email"),
        role=current_admin.get("role", "admin"),
    )
```

```

        is_active=bool(current_admin.get("is_active", True)),
        created_at=current_admin.get("created_at"),
        updated_at=current_admin.get("updated_at")
    )

    @router.post("/logout")
    def admin_logout(current_admin: Dict[str, Any] = Depends(get_current_admin)):
        """
        Confirm logout for authenticated admin session.
        Client clears stored JWT token upon receiving this response.
        """
        return {"message": "Admin logged out successfully"}

```

backend/app/routes/admin.py - Admin KPI Metrics & Analytics

backend/app/routes/admin.py

Python

```

from fastapi import APIRouter, Depends
from typing import Dict, Any
from app.models import DashboardStatsResponse
from app.database import get_admin_dashboard_stats
from app.dependencies import get_current_admin

router = APIRouter(prefix="/api/admin", tags=["Admin Operations"])

@router.get("/stats", response_model=DashboardStatsResponse)
def get_dashboard_stats(current_admin: Dict[str, Any] = Depends(get_current_admin)):
    """
    Protected endpoint to fetch real database metrics for the admin dashboard.
    Counts total products, in-stock items, total repair jobs, and pending repair jobs.
    """
    stats = get_admin_dashboard_stats()
    return DashboardStatsResponse(**stats)

```

backend/app/routes/products.py

Python

```

import os
import uuid
from fastapi import APIRouter, Query, HTTPException, Depends, status, UploadFile, File
from typing import List, Optional, Dict, Any
from app.models import Product, ProductCreate, ProductUpdate, ProductDeleteResponse
from app.database import (
    fetch_all_products,
    fetch_product_by_id,
    create_product,
    update_product,
    delete_product
)
from app.dependencies import get_current_admin

router = APIRouter(prefix="/api/products", tags=["Products"])

# Ensure static upload directory exists
UPLOAD_DIR = os.path.join(os.path.dirname(__file__), "..", "..", "static", "uploads", "products")
os.makedirs(UPLOAD_DIR, exist_ok=True)

# Allowed image MIME types and max size (5MB)
ALLOWED_IMAGE_TYPES = {"image/jpeg", "image/png", "image/webp", "image/gif", "image/jpg"}
MAX_FILE_SIZE = 5 * 1024 * 1024

@router.get("", response_model=List[Product])
def get_products(
    condition: Optional[str] = Query(None, description="'new' or 'refurbished' / 'second_hand'"),
    brand: Optional[str] = Query(None, description="Filter by brand: Samsung, Apple, Vivo, etc."),
    category: Optional[str] = Query(None, description="Filter by category e.g. Smartphones"),
    search: Optional[str] = Query(None, description="Search by title, brand, or model"),
    include_out_of_stock: bool = Query(False, description="Set True for admin panel to see full inventory")
):
    """
    Public catalogue endpoint. By default returns only in-stock items.
    Pass include_out_of_stock=true to retrieve all inventory items for management.
    """
    try:
        products = fetch_all_products(
            condition=condition,
            brand=brand,
            category=category,
            search=search,
            include_out_of_stock=include_out_of_stock
        )
        return products
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/{product_id}", response_model=Product)
def get_product(product_id: str):
    """
    Retrieve single product details by ID.
    """
    product = fetch_product_by_id(product_id)
    if not product:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with ID '{product_id}' was not found in catalogue."
        )
    return product

```

```

@router.post("", response_model=Product, status_code=status.HTTP_201_CREATED)
def add_product(
    payload: ProductCreate,
    current_admin: Dict[str, Any] = Depends(get_current_admin)
):
    """
    Protected Admin endpoint: Create and register a new product in the store database.
    Requires verified admin JWT token.
    """
    try:
        created = create_product(payload.model_dump())
        if not created:
            raise HTTPException(status_code=500, detail="Failed to persist product to database.")
        return created
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Database error during product creation: {str(e)}")

@router.put("/{product_id}", response_model=Product)
def full_update_product(
    product_id: str,
    payload: ProductUpdate,
    current_admin: Dict[str, Any] = Depends(get_current_admin)
):
    """
    Protected Admin endpoint: Update product details.
    Requires verified admin JWT token.
    """
    existing = fetch_product_by_id(product_id)
    if not existing:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with ID '{product_id}' was not found to update."
        )

    update_data = payload.model_dump(exclude_unset=True)
    updated = update_product(product_id, update_data)
    if not updated:
        raise HTTPException(status_code=500, detail="Failed to update product in database.")
    return updated

@router.patch("/{product_id}", response_model=Product)
def partial_update_product(
    product_id: str,
    payload: ProductUpdate,
    current_admin: Dict[str, Any] = Depends(get_current_admin)
):
    """
    Protected Admin endpoint: Quick partial update (price change, stock toggle, etc.).
    Requires verified admin JWT token.
    """
    existing = fetch_product_by_id(product_id)
    if not existing:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with ID '{product_id}' was not found to update."
        )

    update_data = payload.model_dump(exclude_unset=True)
    updated = update_product(product_id, update_data)
    if not updated:
        raise HTTPException(status_code=500, detail="Failed to update product in database.")
    return updated

@router.delete("/{product_id}", response_model=ProductDeleteResponse)
def remove_product(

```

```

    product_id: str,
    current_admin: Dict[str, Any] = Depends(get_current_admin)
):
    """
    Protected Admin endpoint: Delete product from store database.
    Requires verified admin JWT token.
    """
    existing = fetch_product_by_id(product_id)
    if not existing:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with ID '{product_id}' does not exist or has already been deleted."
        )

    success = delete_product(product_id)
    if not success:
        raise HTTPException(status_code=500, detail="Database error during product deletion.")

    return ProductDeleteResponse(
        success=True,
        message=f"Product '{existing.get('title', product_id)}' deleted successfully.",
        id=product_id
    )

@router.post("/upload-image")
async def upload_product_image(
    file: UploadFile = File(...),
    current_admin: Dict[str, Any] = Depends(get_current_admin)
):
    """
    Protected Admin endpoint: Upload product photograph directly from counter camera/storage.
    Validates MIME type, file size <= 5MB, and saves locally to /static/uploads/products/.
    """
    if file.content_type not in ALLOWED_IMAGE_TYPES:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Invalid file format '{file.content_type}'. Please upload JPEG, PNG, or WebP images."
        )

    contents = await file.read()
    if len(contents) > MAX_FILE_SIZE:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Image file size exceeds maximum limit of 5MB."
        )

    # Determine safe file extension
    ext_map = {
        "image/jpeg": ".jpg",
        "image/jpg": ".jpg",
        "image/png": ".png",
        "image/webp": ".webp",
        "image/gif": ".gif"
    }
    ext = ext_map.get(file.content_type, ".jpg")
    safe_filename = f"prod_{uuid.uuid4().hex}{ext}"
    dest_path = os.path.join(UPLOAD_DIR, safe_filename)

    with open(dest_path, "wb") as f:
        f.write(contents)

    relative_url = f"/static/uploads/products/{safe_filename}"
    return {
        "success": True,
        "url": relative_url,
        "filename": safe_filename,
        "size_bytes": len(contents)
    }

```

backend/app/routes/repairs.py - Repair Ticket Intake & Status Transitions

backend/app/routes/repairs.py

Python

```
from fastapi import APIRouter, HTTPException, Path, Query
from typing import Dict, Any, List, Optional
from app.models import RepairJob, RepairJobCreate
from app.database import fetch_repair_job, create_new_repair_job, fetch_all_repairs

router = APIRouter(prefix="/api/repairs", tags=["Repair Tracking"])

@router.get("", response_model=List[RepairJob])
def get_all_repairs(limit: int = Query(50, ge=1, le=200, description="Max records to return")):
    """
    Retrieve list of repair jobs from Amit Mobile Shop service center.
    """
    try:
        return fetch_all_repairs(limit=limit)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Failed to fetch repair jobs: {str(e)}")

@router.get("/{job_sheet_id}", response_model=RepairJob)
def get_repair_status(
    job_sheet_id: str = Path(..., description="Job Sheet ID e.g. AMS-101")
):
    job = fetch_repair_job(job_sheet_id)
    if not job:
        raise HTTPException(
            status_code=404,
            detail=f"Job Sheet '{job_sheet_id}' not found. Please verify the ID on your receipt from Amit Mobile Shop."
        )
    return job

@router.post("", response_model=RepairJob)
def book_repair_job(job_in: RepairJobCreate):
    try:
        created = create_new_repair_job(job_in.dict())
        return created
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Failed to create repair booking: {str(e)}")
```

backend/app/routes/estimator.py - AI Repair Estimation Router

backend/app/routes/estimator.py

Python

```
from fastapi import APIRouter, HTTPException
from app.models import EstimateRequest, EstimateResponse
from app.ai_service import estimate_repair_cost

router = APIRouter(prefix="/api/estimate", tags=["AI Repair Estimator"])

@router.post("", response_model=EstimateResponse)
async def get_repair_estimate(req: EstimateRequest):
    try:
        res = await estimate_repair_cost(
            brand=req.brand,
            model=req.model,
            issue=req.issue,
            notes=req.notes
        )
        return res
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Estimation failed: {str(e)}")
```

backend/seed_admin.py - Command-Line Admin Account Seeding Utility

backend/seed_admin.py

Python

```
import os
import sys
import argparse
import getpass
import uuid
from datetime import datetime

# Ensure backend directory is in path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app.database import get_connection
from app.security import hash_password
from app.config import ADMIN_USERNAME, ADMIN_PASSWORD

def seed_admin(username=None, password=None, email=None, role="owner"):
    target_username = (username or ADMIN_USERNAME or "admin").strip()
    target_password = password or ADMIN_PASSWORD

    if not target_password:
        print(f"=== Creating Admin Account for: {target_username} ===")
        target_password = getpass.getpass("Enter Admin Password: ").strip()
        confirm = getpass.getpass("Confirm Admin Password: ").strip()
        if target_password != confirm:
            print("[-] Error: Passwords do not match.")
            sys.exit(1)

    if len(target_password) < 6:
        print("[-] Error: Password must be at least 6 characters long.")
        sys.exit(1)

    target_email = email or f"{target_username}@amitmobileshop.com"
    hashed = hash_password(target_password)
    now = datetime.now().isoformat()

    conn = get_connection()
    cursor = conn.cursor()

    # Check if admin already exists
    cursor.execute("SELECT id FROM admin_users WHERE LOWER(username) = LOWER(?)", (target_username,))
    existing = cursor.fetchone()

    if existing:
        cursor.execute("""
            UPDATE admin_users
            SET password_hash = ?, email = ?, role = ?, is_active = 1, updated_at = ?
            WHERE id = ?
            """, (hashed, target_email, role, now, existing["id"]))
        print(f"[+] Admin '{target_username}' updated successfully with new password!")
    else:
        new_id = str(uuid.uuid4())
        cursor.execute("""
            INSERT INTO admin_users (id, username, email, password_hash, role, is_active, created_at, updated_at)
            VALUES (?, ?, ?, ?, ?, 1, ?, ?)
            """, (new_id, target_username, target_email, hashed, role, now, now))
        print(f"[+] Admin account '{target_username}' created successfully!")

    conn.commit()
    conn.close()

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Create or update shop admin account.")
    parser.add_argument("--username", "-u", default=None, help="Admin username")
    parser.add_argument("--password", "-p", default=None, help="Admin password (optional; otherwise prompts or uses ADMIN_PASSWORD env)")
```

```
parser.add_argument("--email", "-e", default=None, help="Admin email")
args = parser.parse_args()

seed_admin(username=args.username, password=args.password, email=args.email)
```

backend/run.py - Server Runner Script

backend/run.py

Python

```
import uvicorn
import os
import sys

# Ensure backend directory is on python path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app.config import PORT, HOST

if __name__ == "__main__":
    print(f"Starting Amit Mobile Shop Backend on http://{HOST}:{PORT}")
    uvicorn.run("app.main:app", host=HOST, port=PORT, reload=True)
```

backend/requirements.txt - Python Dependencies List

backend/requirements.txt

Text

```
fastapi>=0.110.0
uvicorn>=0.28.0
pydantic>=2.6.0
python-dotenv>=1.0.1
requests>=2.31.0
httpx>=0.27.0
google-generativeai>=0.4.0
pyjwt>=2.8.0
bcrypt>=4.1.0
```

6. Complete Frontend Source Code

frontend/package.json - Node.js Dependencies & Build Configuration

frontend/package.json

JSON

```
{
  "name": "amit-mobile-shop-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.7.2",
    "clsx": "^2.1.1",
    "lucide-react": "^0.363.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^7.18.4",
    "recharts": "^3.10.1",
    "tailwind-merge": "^2.2.2"
  },
  "devDependencies": {
    "@types/react": "^18.2.66",
    "@types/react-dom": "^18.2.22",
    "@vitejs/plugin-react": "^4.2.1",
    "autoprefixer": "^10.4.19",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.3",
    "vite": "^5.2.0"
  }
}
```

frontend/vite.config.js - Vite Development Server & API Proxy

frontend/vite.config.js

JavaScript

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    host: true,
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
        changeOrigin: true,
        secure: false,
      }
    }
  }
})
```

frontend/tailwind.config.js

JavaScript

```

/** @type {import('tailwindcss').Config} */
export default {
  content: [
    './index.html',
    './src/**/*.{js,ts,jsx,tsx}',
  ],
  theme: {
    extend: {
      colors: {
        primary: {
          50: '#eff6ff',
          100: '#dbeafe',
          200: '#bfdbfe',
          300: '#93c5fd',
          400: '#60a5fa',
          500: '#3b82f6',
          600: '#2563eb', // Royal Blue
          700: '#1d4ed8',
          800: '#1e40af',
          900: '#1e3a8a', // Deep Trust Navy Blue
        },
        gold: {
          50: '#ffffbe',
          100: '#fef3c7',
          500: '#f59e0b',
          600: '#d97706', // Warm Gold/Amber for EMI
          700: '#b45309',
        },
        whatsapp: {
          50: '#f0fdf4',
          100: '#dcfce7',
          500: '#22c55e',
          600: '#16a34a', // Clean WhatsApp Emerald Green
          700: '#15803d',
        },
        space: {
          950: '#060A13', // Deep Luxury Space Navy
          900: '#0B1120',
          800: '#111827',
          700: '#1F2937',
        }
      },
    },
    fontFamily: {
      sans: ['Poppins', 'Noto Sans Devanagari', 'Inter', 'system-ui', '-apple-system', 'sans-serif'],
      hindi: ['Noto Sans Devanagari', 'Poppins', 'sans-serif'],
    },
    boxShadow: {
      'card': '0 4px 20px -2px rgba(0, 0, 0, 0.25), 0 2px 6px -1px rgba(0, 0, 0, 0.15)',
      'card-hover': '0 20px 35px -5px rgba(37, 99, 235, 0.2), 0 8px 16px -3px rgba(0, 0, 0, 0.3)',
      'glow-primary': '0 0 40px -10px rgba(37, 99, 235, 0.45)',
      'glow-gold': '0 0 35px -8px rgba(245, 158, 11, 0.4)',
      'glow-whatsapp': '0 0 30px -8px rgba(22, 163, 74, 0.4)',
    }
  },
  plugins: [],
}

```

frontend/src/main.jsx - React DOM Application Bootstrap

frontend/src/main.jsx

JavaScript/JSX

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

frontend/src/App.jsx - Root Layout, Routing & Route Guards

frontend/src/App.jsx

JavaScript/JSX

```
import React from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import CustomerStorefront from './components/CustomerStorefront';
import ErrorBoundary from './components/ErrorBoundary';
import { LanguageProvider } from './context/LanguageContext';
import { AuthProvider } from './context/AuthContext';
import { CartProvider } from './context/CartContext';
import AdminLoginPage from './components/Admin/AdminLoginPage';
import AdminDashboard from './components/Admin/AdminDashboard';
import ProtectedAdminRoute from './components/Admin/ProtectedAdminRoute';

export default function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <CartProvider>
          <LanguageProvider>
            <Routes>
              { /* Public Customer Storefront with 3D Category Panels */ }
              <Route
                path="/"
                element={
                  <ErrorBoundary>
                    <CustomerStorefront />
                  </ErrorBoundary>
                }
              />

              { /* Owner & Admin Authentication */ }
              <Route path="/admin/login" element={ <AdminLoginPage /> } />
              <Route path="/admin" element={ <Navigate to="/admin/dashboard" replace /> } />
              <Route
                path="/admin/dashboard"
                element={
                  <ProtectedAdminRoute>
                    <AdminDashboard />
                  </ProtectedAdminRoute>
                }
              />

              { /* Fallback to Customer Store */ }
              <Route path="*" element={ <Navigate to="/" replace /> } />
            </Routes>
          </LanguageProvider>
        </CartProvider>
      </AuthProvider>
    </BrowserRouter>
  );
}
```

frontend/src/index.css

CSS

```

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  body {
    @apply bg-slate-50 text-slate-900 font-sans antialiased overflow-x-hidden;
  }

  h1, h2, h3, h4, h5, h6 {
    font-family: 'Poppins', 'Inter', 'Noto Sans Devanagari', sans-serif;
  }
}

/* Professional Retail Button Classes */
.btn-primary {
  @apply inline-flex items-center justify-center gap-2 px-5 py-2.5 rounded-lg bg-primary-700 hover:bg-primary-800 text-white text-sm font-semibold transition-colors duration-150 shadow-sm disabled:opacity-50 disabled:pointer-event none;
}

.btn-secondary {
  @apply inline-flex items-center justify-center gap-2 px-5 py-2.5 rounded-lg bg-white hover:bg-slate-50 text-slate-800 border border-slate-300 text-sm font-semibold transition-colors duration-150 shadow-sm;
}

.btn-whatsapp {
  @apply inline-flex items-center justify-center gap-2 px-5 py-2.5 rounded-lg bg-emerald-600 hover:bg-emerald-700 text-white text-sm font-semibold transition-colors duration-150 shadow-sm;
}

/* Professional Card Classes (Border-driven, subtle elevation) */
.retail-card {
  @apply bg-white border border-slate-200 rounded-xl transition-all duration-200 hover:border-slate-300 hover:shadow-md;
}

.retail-card-interactive {
  @apply bg-white border border-slate-200 rounded-xl transition-all duration-200 hover:border-primary-300 hover:shadow-md;
}

/* Form Controls */
.retail-input {
  @apply w-full bg-white border border-slate-300 rounded-lg px-3.5 py-2 text-sm text-slate-900 placeholder-slate-400 focus:outline-none focus:border-primary-600 focus:ring-1 focus:ring-primary-600 transition-colors;
}

/* Clean Range Slider */
input[type="range"].slider-retail {
  -webkit-appearance: none;
  appearance: none;
  width: 100%;
  height: 6px;
  background: #e2e8f0;
  border-radius: 4px;
  outline: none;
}

input[type="range"].slider-retail::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  width: 18px;
  height: 18px;
}

```

```
border-radius: 50%;
background: #1e3a8a;
border: 2px solid #ffffff;
box-shadow: 0 1px 3px rgba(0, 0, 0, 0.2);
cursor: pointer;
transition: transform 0.1s ease;
}

input[type="range"].slider-retail::-webkit-slider-thumb:hover {
  transform: scale(1.1);
}

/* Clean Scrollbars */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}

::-webkit-scrollbar-track {
  background: #f1f5f9;
}

::-webkit-scrollbar-thumb {
  background: #cbd5e1;
  border-radius: 3px;
}

::-webkit-scrollbar-thumb:hover {
  background: #94a3b8;
}
```

frontend/src/services/api.js

JavaScript

```

import { INITIAL_PRODUCTS, MOCK_REPAIRS, SHOP_INFO } from '../data/mockData';

const BASE_URL = '/api';

export async function fetchProducts(condition = null, brand = null, includeOutOfStock = false, search = null, category = null) {
  try {
    let url = `${BASE_URL}/products`;
    const params = new URLSearchParams();
    if (condition) params.append('condition', condition);
    if (brand && brand.toLowerCase() !== 'all' && brand !== 'All Brands') params.append('brand', brand);
    if (category && category.toLowerCase() !== 'all') params.append('category', category);
    if (search && search.trim()) params.append('search', search.trim());
    if (includeOutOfStock) params.append('include_out_of_stock', 'true');

    if (params.toString()) {
      url += `?${params.toString()}`;
    }

    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), 3500);

    const res = await fetch(url, { signal: controller.signal });
    clearTimeout(timeoutId);

    if (!res.ok) throw new Error(`HTTP error ${res.status}`);
    const data = await res.json();
    if (Array.isArray(data)) {
      return data;
    }
    console.warn('[API] Products response was not an array:', data);
    throw new Error('Non-array response from API');
  } catch (err) {
    console.warn('[API] Products fetch failed or timed out, using fallback:', err);
    let filtered = Array.isArray(INITIAL_PRODUCTS) ? [...INITIAL_PRODUCTS] : [];
    if (condition === 'new') {
      filtered = filtered.filter(p => p.condition === 'new');
    } else if (condition === 'refurbished' || condition === 'second_hand') {
      filtered = filtered.filter(p => p.condition !== 'new');
    }
    if (brand && brand !== 'All Brands' && brand.toLowerCase() !== 'all') {
      filtered = filtered.filter(p => (p.brand || '').toLowerCase() === brand.toLowerCase());
    }
    if (!includeOutOfStock) {
      filtered = filtered.filter(p => p.in_stock);
    }
    return filtered;
  }
}

export async function fetchRepairStatus(jobSheetId) {
  const cleanId = (jobSheetId || '').trim().toUpperCase();
  try {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), 3500);
    const res = await fetch(`${BASE_URL}/repairs/${cleanId}`, { signal: controller.signal });
    clearTimeout(timeoutId);

    if (!res.ok) {
      if (res.status === 404) {
        throw new Error(`Job Sheet "${cleanId}" not found. Please check receipt.`);
      }
      throw new Error(`Tracking server error: ${res.status}`);
    }
  }
}

```

```

    return await res.json();
  } catch (err) {
    console.warn('[API] Repair tracking fetch failed, checking local database:', err);
    if (MOCK_REPAIRS && MOCK_REPAIRS[cleanId]) {
      return MOCK_REPAIRS[cleanId];
    }
    throw err;
  }
}

export async function calculateEstimate(brand, model, issue, notes = '') {
  try {
    const res = await fetch(`${BASE_URL}/estimate`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ brand, model, issue, notes })
    });
    if (!res.ok) throw new Error(`Estimate API returned ${res.status}`);
    return await res.json();
  } catch (err) {
    console.warn('[API] AI Estimator server error, calculating local heuristic:', err);
    const issueLower = issue.toLowerCase();
    let minCost = 1400;
    let maxCost = 2200;
    let time = "1 - 2 Hours";
    let quality = "High Quality AAA+ Tested Folder with 3 Months Warranty";

    if (issueLower.includes('battery')) {
      minCost = 850;
      maxCost = 1400;
      time = "30 - 45 Minutes";
      quality = "Certified Grade A+ Battery with 6 Months Guarantee";
    } else if (issueLower.includes('charg') || issueLower.includes('port')) {
      minCost = 450;
      maxCost = 750;
      time = "30 Minutes";
      quality = "Original Type-C Sub-board with Fast Charge";
    } else if (issueLower.includes('water')) {
      minCost = 900;
      maxCost = 1800;
      time = "3 - 4 Hours";
      quality = "Ultrasonic Motherboard De-oxidation & Circuit Test";
    } else if (issueLower.includes('mic') || issueLower.includes('speaker')) {
      minCost = 350;
      maxCost = 650;
      time = "30 Minutes";
      quality = "Original Sound Chamber Component";
    }

    const waText = encodeURIComponent(
      `Hello Amit Mobile Shop, I need an estimate for my ${brand} ${model} (${issue}). Estimated cost: ₹${minCost} - ₹${maxCost}. Can I visit today?`
    );

    return {
      brand,
      model,
      issue,
      estimated_min_cost: minCost,
      estimated_max_cost: maxCost,
      turnaround_time: time,
      part_quality: quality,
      confidence: "Verified Store Benchmark",
      ai_generated: false,
      summary: `Estimated repair for ${brand} ${model} (${issue}) is ₹${minCost} - ₹${maxCost}.`,
      technician_tip: "Visit Amit Mobile Shop (Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312) for counter diagnosis.",
      breakdown: {
        part_cost_min: minCost - 300,
        part_cost_max: maxCost - 300,
      }
    };
  }
}

```

```

        service_charge: 300,
        estimated_min_total: minCost,
        estimated_max_total: maxCost,
        estimated_turnaround_time: time,
        warranty_provided: "Up to 6 Months Warranty + Instant Testing at Counter"
    },
    whatsapp_link: `https://wa.me/${SHOP_INFO.whatsapp}?text=${waText}`
  };
}
}

// --- Admin Authentication & Dashboard API ---

export function getStoredToken() {
  return localStorage.getItem('ams_admin_token');
}

export function setStoredToken(token) {
  if (token) {
    localStorage.setItem('ams_admin_token', token);
  } else {
    localStorage.removeItem('ams_admin_token');
  }
}

export function clearStoredToken() {
  localStorage.removeItem('ams_admin_token');
}

export async function adminLogin(username, password) {
  const res = await fetch(`${BASE_URL}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username, password })
  });

  if (!res.ok) {
    const errorData = await res.json().catch(() => ({}));
    throw new Error(errorData.detail || 'Invalid username or password. Please try again.');
```

```

    if (token) {
      await fetch(`${BASE_URL}/auth/logout`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${token}` }
      });
    }
  } catch (err) {
    console.warn('[API] Logout request error:', err);
  } finally {
    clearStoredToken();
  }
  return true;
}

export async function fetchAdminStats() {
  const token = getStoredToken();
  if (!token) throw new Error('Authentication required');

  const res = await fetch(`${BASE_URL}/admin/stats`, {
    headers: {
      'Authorization': `Bearer ${token}`
    }
  });

  if (!res.ok) {
    if (res.status === 401) {
      clearStoredToken();
    }
    throw new Error(`Failed to fetch admin stats: ${res.status}`);
  }

  return await res.json();
}

// --- Product Management CRUD Services ---

export async function fetchProductById(productId) {
  const res = await fetch(`${BASE_URL}/products/${productId}`);
  if (!res.ok) {
    if (res.status === 404) {
      throw new Error('Product not found.');
```

```

    throw new Error(msg || `Failed to create product (${res.status})`);
  }

  return await res.json();
}

export async function updateProduct(productId, productData) {
  const token = getStoredToken();
  if (!token) throw new Error('Authentication required to update product.');
```

```

  const res = await fetch(`${BASE_URL}/products/${productId}`, {
    method: 'PUT',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`
    },
    body: JSON.stringify(productData)
  });

  if (!res.ok) {
    if (res.status === 401) {
      clearStoredToken();
      throw new Error('Session expired or unauthorized. Please log in again.');
```

```

    }
    const errData = await res.json().catch(() => ({}));
    let msg = errData.detail;
    if (Array.isArray(msg)) {
      msg = msg.map(m => m.msg || m.message).join(', ');
    }
    throw new Error(msg || `Failed to update product (${res.status})`);
  }

  return await res.json();
}

export async function patchProduct(productId, patchData) {
  const token = getStoredToken();
  if (!token) throw new Error('Authentication required to update product.');
```

```

  const res = await fetch(`${BASE_URL}/products/${productId}`, {
    method: 'PATCH',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`
    },
    body: JSON.stringify(patchData)
  });

  if (!res.ok) {
    if (res.status === 401) {
      clearStoredToken();
      throw new Error('Session expired or unauthorized. Please log in again.');
```

```

    }
    const errData = await res.json().catch(() => ({}));
    let msg = errData.detail;
    if (Array.isArray(msg)) {
      msg = msg.map(m => m.msg || m.message).join(', ');
    }
    throw new Error(msg || `Failed to update product (${res.status})`);
  }

  return await res.json();
}

export async function deleteProduct(productId) {
  const token = getStoredToken();
  if (!token) throw new Error('Authentication required to delete product.');
```

```

  const res = await fetch(`${BASE_URL}/products/${productId}`, {
    method: 'DELETE',

```

```

    headers: {
      'Authorization': `Bearer ${token}`
    }
  });

  if (!res.ok) {
    if (res.status === 401) {
      clearStoredToken();
      throw new Error('Session expired or unauthorized. Please log in again.');
```

```

    }
    const errData = await res.json().catch(() => ({}));
    throw new Error(errData.detail || `Failed to delete product (${res.status})`);
  }

  return await res.json();
}

export async function uploadProductImage(file) {
  const token = getStoredToken();
  if (!token) throw new Error('Authentication required to upload image.');
```

```

  const formData = new FormData();
  formData.append('file', file);

  const res = await fetch(`${BASE_URL}/products/upload-image`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`
    },
    body: formData
  });
```

```

  if (!res.ok) {
    if (res.status === 401) {
      clearStoredToken();
      throw new Error('Session expired or unauthorized. Please log in again.');
```

```

    }
    const errData = await res.json().catch(() => ({}));
    throw new Error(errData.detail || `Image upload failed (${res.status})`);
  }

  return await res.json();
}

```

frontend/src/context/AuthContext.jsx

JavaScript/JSX

```

import React, { createContext, useContext, useState, useEffect } from 'react';
import { adminLogin, getCurrentAdmin, adminLogout, getStoredToken, clearStoredToken } from '../services/api';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [admin, setAdmin] = useState(null);
  const [token, setToken] = useState(getStoredToken());
  const [isLoading, setIsLoading] = useState(true);
  const [authError, setAuthError] = useState(null);

  // Validate existing token on mount
  useEffect(() => {
    async function checkExistingAuth() {
      const storedToken = getStoredToken();
      if (!storedToken) {
        setIsLoading(false);
        return;
      }

      try {
        const adminProfile = await getCurrentAdmin();
        setAdmin(adminProfile);
        setToken(storedToken);
      } catch (err) {
        console.warn('[Auth] Session validation failed:', err.message);
        clearStoredToken();
        setAdmin(null);
        setToken(null);
      } finally {
        setIsLoading(false);
      }
    }

    checkExistingAuth();
  }, []);

  const login = async (username, password) => {
    setIsLoading(true);
    setAuthError(null);
    try {
      const data = await adminLogin(username, password);
      setToken(data.access_token);
      setAdmin(data.admin);
      return { success: true };
    } catch (err) {
      setAuthError(err.message || 'Login failed');
      return { success: false, error: err.message };
    } finally {
      setIsLoading(false);
    }
  };

  const logout = async () => {
    setIsLoading(true);
    try {
      await adminLogout();
    } catch (err) {
      console.warn('[Auth] Error during logout:', err);
    } finally {
      clearStoredToken();
      setAdmin(null);
      setToken(null);
      setAuthError(null);
    }
  };

```

```

        setIsLoading(false);
    }
};

const clearError = () => setAuthError(null);

return (
  <AuthContext.Provider
    value={{
      admin,
      token,
      isAuthenticated: !!admin,
      isLoading,
      authError,
      login,
      logout,
      clearError
    }}
  >
    {children}
  </AuthContext.Provider>
);
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) {
    return {
      admin: null,
      token: null,
      isAuthenticated: false,
      isLoading: false,
      authError: null,
      login: async () => ({ success: false }),
      logout: async () => {},
      clearError: () => {}
    };
  }
  return ctx;
}

```

frontend/src/context/LanguageContext.jsx

JavaScript/JSX

```

import React, { createContext, useContext, useState, useEffect } from 'react';

const LanguageContext = createContext();

export const translations = {
  hi: {
    // Top Bar & Navbar
    shop_title: "Amit Mobile Shop",
    shop_subtitle: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312 • नया फोन, सेकंड हैंड फोन व 1 घंटे में रिपेयरिंग",
    location_badge: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312",
    timing: "रोजाना सुबह 9:00 से रात 8:30 बजे तक",
    lang_toggle_btn: "English",
    whatsapp_chat: "व्हाट्सएप पर बात करें",
    call_now: "कॉल करें",

    // Main 2-Hub Buttons
    hub_buying: "1. मोबाइल खरीदें (Buying Hub)",
    hub_repairing: "2. मोबाइल रिपेयरिंग (Repairing Hub)",

    // Hero Section
    hero_badge: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312 - सबसे भरोसेमंद मोबाइल स्टोर",
    hero_title: "नया मोबाइल खरीदें या फोन 1 घंटे में ठीक करवाएं",
    hero_subtitle: "बजाज फिनसर्व, TVS क्रेडिट और सैमसंग फाइनेंस+ पर बिना किसी इंडेंट के आसान किश्तों (0% EMI) पर फोन ले जाएं। रिपेयरिंग का लाइव स्टेटस रसीद नंबर से देखें।",
    hero_cta_buy: "नया व सेकंड हैंड फोन देखें",
    hero_cta_repair: "रिपेयरिंग खर्चा और स्टेटस देखें",

    // Value Prop Chips
    trust_emi: "0% आसान किश्तें (EMI)",
    trust_emi_desc: "बजाज, TVS, सैमसंग फाइनेंस",
    trust_repair: "1 घंटे में रिपेयर",
    trust_repair_desc: "स्क्रीन व बैटरी तुरंत बदलें",
    trust_warranty: "पक्की दुकान वारंटी",
    trust_warranty_desc: "32-पॉइंट चेकिंग + पक्का बिल",
    trust_support: "सीधा फोन सपोर्ट",
    trust_support_desc: "Amit Mobile Shop सपोर्ट",

    // Buying Hub
    buying_title: "Amit Mobile Shop - Buying Hub",
    buying_desc: "नए ओरिजिनल 5G स्मार्टफोन या दुकान की वारंटी वाले प्रमाणित सेकंड-हैंड फोन चुनें।",
    tab_new: "टैब A: नया फोन खरीदें (New)",
    tab_refurb: "टैब B: सेकंड-हैंड फोन खरीदें (Second-Hand)",
    search_new_placeholder: "नया फोन खोजें (उदा. Galaxy S24, Vivo V30)...",
    search_refurb_placeholder: "पुराना फोन खोजें (उदा. iPhone 13, OnePlus)...",
    all_brands: "सभी ब्रांड",
    phones_available: "फोन उपलब्ध हैं",
    no_phones_found: "कोई फोन नहीं मिला",
    reset_filters: "फ़िल्टर रीसेट करें",
    badge_brand_new: "★ सीलबंद नया",
    badge_like_new: "✓ एकदम नए जैसा (Like New)",
    badge_good: "✓ बढ़िया कंडीशन (Good)",
    emi_calc_btn: "EMI किश्त जानें",
    buy_whatsapp_btn: "व्हाट्सएप पर बुक करें",
    offer_price: "ऑफर कीमत:",

    // EMI Spotlight
    emi_banner_title: "0% ब्याज और आसान किश्तों की सुविधा - Amit Mobile Shop",
    emi_banner_desc: "पूरा पैसा एक बार में देने की जरूरत नहीं। केवल आधार कार्ड, पैन कार्ड या बैंक पासबुक लाकर 15 मिनट में फोन ले जाएं।",
    open_emi_calc: "किश्त कैलकुलेटर खोलें",
    bajaj_title: "बजाज फिनसर्व (Bajaj)",
    bajaj_badge: "0% ब्याज",
    bajaj_desc: "पुराने कार्ड धारकों और नए ग्राहकों के लिए तुरंत अप्रूवल।",
    tvs_title: "TVS क्रेडिट (TVS)",

```

```

    tvs_badge: "आसान अप्रूवल",
    tvs_desc: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312 व आसपास के ग्रामीण भाइयों के लिए कम से कम डाउन पेमेंट।",
    samsung_title: "सैमसंग फाइनेंस+ (Samsung)",
    samsung_badge: "100% पेपरलेस",
    samsung_desc: "सैमसंग फोन के लिए आधार OTP से 5 मिनट में डिजिटल लोन।",

    // EMI Calculator Modal
    modal_calc_title: "EMI किश्त कैलकुलेटर - Amit Mobile Shop",
    modal_device_price: "फोन की कीमत",
    modal_est_monthly: "अनुमानित मासिक किश्त",
    modal_down_payment: "डाउन पेमेंट (शुरुआती जमा)",
    modal_zero_down: "₹0 (शून्य डाउन पेमेंट)",
    modal_tenure: "किश्त की अवधि (महीने)",
    modal_months: "महीने",
    modal_choose_partner: "फाइनेंस कंपनी चुनें",
    modal_doc_tip: "दुकान (Amit Mobile Shop, Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312) पर केवल आधार कार्ड, पैन कार्ड और बैंक पासबुक साथ लाएं। तुरंत अप्रूवल मिलेगा।",
    modal_apply_whatsapp: "इस EMI प्लान के लिए व्हाट्सएप करें",

    // Repairing Hub
    repair_title: "Amit Mobile Shop - Repairing Hub",
    repair_desc: "दुकान आने से पहले घर बैठे खर्चे और समय का तुरंत पता लगाएं या अपनी रसीद से रिपेयरिंग स्टेटस ट्रैक करें।",
    subtab_estimator: "खर्चे और समय का अनुमान (Price Estimator)",
    subtab_tracker: "रसीद नंबर से स्थिति देखें (Job Sheet Tracker)",
    express_guarantee_1: "1 घंटे में कॉम्बो / स्क्रीन रिप्लेसमेंट",
    express_guarantee_2: "माइक्रोस्कोप से मदरबोर्ड व IC रिपेयरिंग",
    express_guarantee_3: "3 से 6 महीने की दुकान वारंटी",

    // Estimator Form
    step_1_brand: "1. मोबाइल का ब्रांड चुनें",
    step_2_model: "2. मोबाइल का मॉडल लिखें",
    step_3_issue: "3. फोन में क्या खराबी है चुनें",
    step_4_notes: "4. कोई और जानकारी (वैकल्पिक)",
    calc_estimate_btn: "खर्चा और समय का अनुमान लगाएं",
    calculating: "अनुमान लगाया जा रहा है...",
    est_cost_label: "अनुमानित खर्चा",
    est_time_label: "रिपेयरिंग में लगा समय",
    est_quality_label: "पार्ट्स की कालिटी",
    est_warranty_label: "दुकान की वारंटी",
    tested_grade: "ओरिजिनल टेस्टेड ग्रेड",
    tech_tip_label: "Amit Mobile Shop सलाह:",
    show_breakdown: "खर्चे का ब्यौरा देखें (सामान + मजदूरी)",
    hide_breakdown: "ब्यौरा छिपाएं",
    part_cost_label: "सामान (स्पेयर पार्ट) का खर्चा:",
    labor_cost_label: "कारीगरी व सर्विस फीस:",
    book_repair_wa: "यह रिपेयर व्हाट्सएप पर बुक करें और दुकान आएं",

    // Job Sheet Tracker
    tracker_heading: "फोन रिपेयरिंग का लाइव स्टेटस देखें - Amit Mobile Shop",
    tracker_subheading: "अपनी जमा रसीद पर लिखा जॉब शीट नंबर दर्ज करें (उदा. AMS-101)",
    tracker_input_placeholder: "जॉब शीट नंबर लिखें (उदा. AMS-101)...",
    tracker_btn: "स्टेटस चेक करें",
    quick_test_chips: "डेमो रसीद नंबर:",
    stage_received: "दुकान पर जमा",
    stage_received_desc: "फोन काउंटर पर प्राप्त हुआ",
    stage_in_repair: "रिपेयरिंग जारी",
    stage_in_repair_desc: "कारीगर फोन ठीक कर रहे हैं",
    stage_ready: "ले जाने के लिए तैयार",
    stage_ready_desc: "टेस्टिंग पूरी, दुकान काउंटर पर तैयार",
    stage_delivered: "ग्राहक को सौंप दिया",
    stage_delivered_desc: "वारंटी बिल के साथ दिया गया",
    cust_name_label: "ग्राहक का नाम:",
    issue_label: "शिकायत / खराबी:",
    est_bill_label: "अनुमानित बिल:",
    handled_by_label: "कारीगर:",
    tech_note_label: "कारीगर का नोट:",
    track_wa_inquiry: "इस रसीद के बारे में व्हाट्सएप पर बात करें",

    // Contact Bar & Footer

```

```

quick_call: "सीधा फोन लगाएं",
footer_tagline: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312 - सबसे भरोसेमंद मोबाइल शोरूम व एक्सप्रेस सर्विस सेंटर",
},
footer_address_title: "दुकान का पूरा पता व समय",
footer_address: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312",
footer_directions: "गूगल मैप्स पर दुकान का सही रास्ता देखें",
footer_contacts_title: "दुकान के फोन नंबर",
footer_rights: "सर्वाधिकार सुरक्षित। Amit Mobile Shop.",
},
en: {
  // Top Bar & Navbar
  shop_title: "Amit Mobile Shop",
  shop_subtitle: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312 • New Phones & 1-Hour Repairs",
  location_badge: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312",
  timing: "Daily: 9:00 AM - 8:30 PM",
  lang_toggle_btn: "हिंदी",
  whatsapp_chat: "WhatsApp Chat",
  call_now: "Call Store",

  // Main 2-Hub Buttons
  hub_buying: "1. Buying Hub (Phones)",
  hub_repairing: "2. Repairing Hub (Service)",

  // Hero Section
  hero_badge: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312 - Most Trusted Store",
  hero_title: "Buy Latest Smartphones or Get 1-Hour Express Repairs",
  hero_subtitle: "Walk away with your dream phone on Bajaj Finserv, TVS Credit & Samsung Finance+ 0% EMI. Or track your device repair live with your receipt ID.",
  hero_cta_buy: "Explore Phones & EMI Plans",
  hero_cta_repair: "Check Repair Cost & Track Job",

  // Value Prop Chips
  trust_emi: "0% Easy EMI Finance",
  trust_emi_desc: "Bajaj, TVS & Samsung",
  trust_repair: "1-Hour Express Service",
  trust_repair_desc: "Display & battery while you wait",
  trust_warranty: "Verified Shop Warranty",
  trust_warranty_desc: "32-Point check + cash bill",
  trust_support: "Direct Phone Support",
  trust_support_desc: "Amit Mobile Shop Support",

  // Buying Hub
  buying_title: "Amit Mobile Shop - Buying Hub",
  buying_desc: "Browse brand new 5G phones with official warranty or tested pre-owned refurbished smartphones.",
  tab_new: "Tab A: Buy New Phone",
  tab_refurb: "Tab B: Buy Second-Hand Phone",
  search_new_placeholder: "Search new phones (e.g. Galaxy S24, Vivo V30)...",
  search_refurb_placeholder: "Search refurbished phones (e.g. iPhone 13, OnePlus)...",
  all_brands: "All Brands",
  phones_available: "Phones Available",
  no_phones_found: "No Phones Found",
  reset_filters: "Reset Filters",
  badge_brand_new: "★ Brand New",
  badge_like_new: "✓ Like New Refurbished",
  badge_good: "✓ Good Condition Pre-Owned",
  emi_calc_btn: "Check EMI",
  buy_whatsapp_btn: "Book on WhatsApp",
  offer_price: "Offer Price:",

  // EMI Spotlight
  emi_banner_title: "0% Interest & Easy EMI Available at Amit Mobile Shop",
  emi_banner_desc: "No need for full upfront cash. Bring Aadhaar Card, PAN Card or Bank Passbook to get instant 15-minute financing approval.",
  open_emi_calc: "Open EMI Calculator",
  bajaj_title: "Bajaj Finserv",
  bajaj_badge: "0% Interest",
  bajaj_desc: "Instant digital approval for existing EMI Card holders & new buyers.",
  tvs_title: "TVS Credit",
  tvs_badge: "Easy Approval",
  tvs_desc: "Lowest down payment schemes crafted for Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312 & rural cu

```

```

stomers.",
    samsung_title: "Samsung Finance+",
    samsung_badge: "100% Paperless",
    samsung_desc: "5-minute paperless loan via Aadhaar OTP on Samsung Galaxy phones.",

    // EMI Calculator Modal
    modal_calc_title: "EMI Financing Calculator - Amit Mobile Shop",
    modal_device_price: "Device Price",
    modal_est_monthly: "Est. Monthly EMI",
    modal_down_payment: "Down Payment",
    modal_zero_down: "₹0 (Zero Down Payment)",
    modal_tenure: "Loan Tenure (Months)",
    modal_months: "Months",
    modal_choose_partner: "Select Financing Partner",
    modal_doc_tip: "Bring your Aadhaar Card, PAN Card, and Bank Passbook to Amit Mobile Shop (Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312) for instant approval.",
    modal_apply_whatsapp: "Apply for this EMI via WhatsApp",

    // Repairing Hub
    repair_title: "Amit Mobile Shop - Repairing Hub",
    repair_desc: "Transparent repair prices and turnaround time estimations before visiting, plus receipt tracking.",
    subtab_estimator: "Repair Price Estimator",
    subtab_tracker: "Live Job Sheet Tracker",
    express_guarantee_1: "1-Hour Express Combo / Display Replacement",
    express_guarantee_2: "Microscope Chip-Level Motherboard & IC Repair",
    express_guarantee_3: "3 to 6 Months Warranty + Bill",

    // Estimator Form
    step_1_brand: "1. Select Device Brand",
    step_2_model: "2. Enter Exact Model Name",
    step_3_issue: "3. Choose the Problem",
    step_4_notes: "4. Additional Details (Optional)",
    calc_estimate_btn: "Calculate Repair Cost & Turnaround Time",
    calculating: "Calculating estimate...",
    est_cost_label: "Estimated Cost",
    est_time_label: "Est. Repair Time",
    est_quality_label: "Part Quality",
    est_warranty_label: "Shop Warranty",
    tested_grade: "Original Tested Grade",
    tech_tip_label: "Amit Mobile Shop Tip:",
    show_breakdown: "Show Cost Breakdown (Parts vs Labor)",
    hide_breakdown: "Hide Cost Breakdown",
    part_cost_label: "Estimated Spare Part Cost:",
    labor_cost_label: "Technician Bench & Service Fee:",
    book_repair_wa: "Book this repair on WhatsApp & visit store",

    // Job Sheet Tracker
    tracker_heading: "Track Your Phone Repair Status - Amit Mobile Shop",
    tracker_subheading: "Enter the Job Sheet ID on your deposit receipt (e.g. AMS-101)",
    tracker_input_placeholder: "Enter Job Sheet ID (e.g. AMS-101)...",
    tracker_btn: "Track Status",
    quick_test_chips: "Quick Demo Job Sheets:",
    stage_received: "Received at Store",
    stage_received_desc: "Device checked in at shop desk",
    stage_in_repair: "In Repair",
    stage_in_repair_desc: "Technician bench repair underway",
    stage_ready: "Ready for Pickup",
    stage_ready_desc: "Quality verified, ready for pickup",
    stage_delivered: "Delivered",
    stage_delivered_desc: "Collected with warranty receipt",
    cust_name_label: "Customer Name:",
    issue_label: "Reported Problem:",
    est_bill_label: "Estimated Bill:",
    handled_by_label: "Handled By:",
    tech_note_label: "Technician Note:",
    track_wa_inquiry: "Chat on WhatsApp regarding this Job Sheet",

    // Contact Bar & Footer
    quick_call: "Call Store Now",

```

```

    footer_tagline: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312 - premier mobile showroom and express repair center.",
    footer_address_title: "Store Location & Hours",
    footer_address: "Kuk Nagar Grint Rd, Khorare, Uttar Pradesh 271312",
    footer_directions: "Get Directions on Google Maps",
    footer_contacts_title: "Store Contacts",
    footer_rights: "All rights reserved. Amit Mobile Shop.",
  }
};

export function LanguageProvider({ children }) {
  const [language, setLanguageState] = useState(() => {
    try {
      return localStorage.getItem('ams_language') || 'hi';
    } catch {
      return 'hi';
    }
  });
  const [hasChosenLanguage, setHasChosenLanguage] = useState(true);

  useEffect(() => {
    try {
      const saved = localStorage.getItem('ams_language');
      if (saved === 'hi' || saved === 'en') {
        setLanguageState(saved);
      }
      setHasChosenLanguage(true);
    } catch (e) {
      console.warn('LocalStorage error:', e);
      setHasChosenLanguage(true);
    }
  }, []);

  const selectLanguage = (lang) => {
    setLanguageState(lang);
    try {
      localStorage.setItem('ams_language', lang);
    } catch (e) {
      console.warn('Failed to save language:', e);
    }
    setHasChosenLanguage(true);
  };

  const toggleLanguage = () => {
    const next = language === 'hi' ? 'en' : 'hi';
    selectLanguage(next);
  };

  const t = (key) => {
    const currentDict = translations[language] || translations.hi;
    return currentDict[key] || translations.en[key] || key;
  };

  return (
    <LanguageContext.Provider value={{ language, setLanguage: selectLanguage, toggleLanguage, hasChosenLanguage, t }}>
      {children}
    </LanguageContext.Provider>
  );
}

export function useLanguage() {
  const ctx = useContext(LanguageContext);
  if (!ctx) {
    return {
      language: 'hi',
      setLanguage: () => {},
      toggleLanguage: () => {},
      hasChosenLanguage: true,
      t: (key) => (translations.hi && translations.hi[key]) || (translations.en && translations.en[key]) || key
    };
  }
}

```

```
};  
}  
return ctx;  
}
```

frontend/src/context/CartContext.jsx

JavaScript/JSX

```

import React, { createContext, useContext, useState, useEffect } from 'react';

const CartContext = createContext();

export function CartProvider({ children }) {
  const [cart, setCart] = useState(() => {
    try {
      const saved = localStorage.getItem('ams_cart');
      return saved ? JSON.parse(saved) : [];
    } catch {
      return [];
    }
  });
  const [isCartOpen, setIsCartOpen] = useState(false);

  useEffect(() => {
    try {
      localStorage.setItem('ams_cart', JSON.stringify(cart));
    } catch (e) {
      console.warn('Failed to save cart to localStorage', e);
    }
  }, [cart]);

  const addToCart = (product, colorVariant = null) => {
    const selectedColor = colorVariant || product.color || 'Standard';
    const cartItemId = `${product.id}-${selectedColor}`;

    setCart((prev) => {
      const existing = prev.find((item) => item.cartItemId === cartItemId);
      if (existing) {
        return prev.map((item) =>
          item.cartItemId === cartItemId
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      }
      return [
        ...prev,
        {
          cartItemId,
          id: product.id,
          title: product.title,
          brand: product.brand,
          model: product.model,
          price: Number(product.price) || 0,
          original_price: product.original_price ? Number(product.original_price) : null,
          image_url: product.image_url,
          condition: product.condition,
          color: selectedColor,
          ram_storage: product.ram_storage || 'Standard Specs',
          quantity: 1
        }
      ];
    });

    setIsCartOpen(true);
  };

  const removeFromCart = (cartItemId) => {
    setCart((prev) => prev.filter((item) => item.cartItemId !== cartItemId));
  };

  const updateQuantity = (cartItemId, delta) => {
    setCart((prev) => {

```

```

    prev
    .map((item) => {
      if (item.cartItemId === cartItemId) {
        const nextQty = item.quantity + delta;
        return nextQty > 0 ? { ...item, quantity: nextQty } : null;
      }
      return item;
    })
    .filter(Boolean)
  );
};

const clearCart = () => {
  setCart([]);
};

const cartCount = cart.reduce((sum, item) => sum + item.quantity, 0);
const cartTotal = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);
const cartMonthlyEmi = Math.round(cartTotal / 12);

return (
  <CartContext.Provider
    value={{
      cart,
      addToCart,
      removeFromCart,
      updateQuantity,
      clearCart,
      cartCount,
      cartTotal,
      cartMonthlyEmi,
      isCartOpen,
      setIsCartOpen,
      openCart: () => setIsCartOpen(true),
      closeCart: () => setIsCartOpen(false)
    }}
  >
    {children}
  </CartContext.Provider>
);
}

export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) {
    return {
      cart: [],
      addToCart: () => {},
      removeFromCart: () => {},
      updateQuantity: () => {},
      clearCart: () => {},
      cartCount: 0,
      cartTotal: 0,
      cartMonthlyEmi: 0,
      isCartOpen: false,
      setIsCartOpen: () => {},
      openCart: () => {},
      closeCart: () => {}
    };
  }
  return ctx;
}

```

frontend/src/components/BuyingHub/BuyingHub.jsx

JavaScript/JSX

```

import React, { useState, useEffect } from 'react';
import { Smartphone, Sparkles, Search, RefreshCw, Layers, ShieldCheck } from 'lucide-react';
import ProductCard from './ProductCard';
import EmiCalculatorModal from './EmiCalculatorModal';
import LiveEmiCalculatorWidget from './LiveEmiCalculatorWidget';
import EMIBadges from './EMIBadges';
import { fetchProducts } from '../../services/api';
import { useLanguage } from '../../context/LanguageContext';
import { INITIAL_PRODUCTS } from '../../data/mockData';

const BRANDS = ['All Brands', 'Samsung', 'Apple', 'OnePlus', 'Vivo', 'Realme', 'Xiaomi'];

export default function BuyingHub() {
  const { t, language } = useLanguage();
  const [buyingTab, setBuyingTab] = useState('new'); // 'new' or 'refurbished'
  const [products, setProducts] = useState(INITIAL_PRODUCTS);
  const [loading, setLoading] = useState(false);
  const [selectedBrand, setSelectedBrand] = useState('All Brands');
  const [searchQuery, setSearchQuery] = useState('');
  const [selectedForEmi, setSelectedForEmi] = useState(null);

  useEffect(() => {
    loadProducts();
  }, [buyingTab, selectedBrand]);

  const loadProducts = async () => {
    try {
      const brandParam = selectedBrand === 'All Brands' ? null : selectedBrand;
      const data = await fetchProducts(buyingTab, brandParam);
      if (Array.isArray(data) && data.length > 0) {
        setProducts(data);
      } else if (Array.isArray(data)) {
        setProducts(data);
      } else {
        setProducts(INITIAL_PRODUCTS);
      }
    } catch (err) {
      console.warn('[BuyingHub] Failed to load products from API, retaining fallback:', err);
    }
  };

  const filteredProducts = (products || []).filter((p) => {
    if (!p) return false;
    if (!searchQuery) return true;
    const q = searchQuery.toLowerCase();
    const title = (p.title || '').toLowerCase();
    const brand = (p.brand || '').toLowerCase();
    const desc = (p.description || '').toLowerCase();
    return title.includes(q) || brand.includes(q) || desc.includes(q);
  });

  return (
    <section className="space-y-8">

      {/* Product Section Header & Sub-Tabs */}
      <div className="bg-white border border-slate-200 rounded-xl p-5 sm:p-6 shadow-xs flex flex-col md:flex-row it
ems-start md:items-center justify-between gap-4">
        <div>
          <div className="flex items-center gap-2 mb-1">
            <Smartphone className="w-5 h-5 text-primary-800" />
            <h2 className="text-xl sm:text-2xl font-black text-slate-900 font-['Poppins']">
              {t('buying_title')}
            </h2>
          </div>
        </div>

```

```

        <p className="text-xs sm:text-sm text-slate-500 font-normal">
            {t('buying_desc')}
        </p>
    </div>

    {/* Primary Sub-Tabs: Brand New vs Certified Pre-Owned */}
    <div className="inline-flex p-1 rounded-lg bg-slate-100 border border-slate-200 w-full md:w-auto">
        <button
            onClick={() => setBuyingTab('new')}
            className={`flex-1 md:flex-none flex items-center justify-center gap-2 px-5 py-2 rounded-md font-semibo
ld text-xs sm:text-sm transition-all ${
                buyingTab === 'new'
                ? 'bg-white text-slate-900 shadow-xs'
                : 'text-slate-600 hover:text-slate-900'
            }`}
        >
            <span>{t('tab_new')}</span>
        </button>

        <button
            onClick={() => setBuyingTab('refurbished')}
            className={`flex-1 md:flex-none flex items-center justify-center gap-2 px-5 py-2 rounded-md font-semibo
ld text-xs sm:text-sm transition-all ${
                buyingTab === 'refurbished'
                ? 'bg-white text-slate-900 shadow-xs'
                : 'text-slate-600 hover:text-slate-900'
            }`}
        >
            <RefreshCw className="w-3.5 h-3.5 text-slate-500" />
            <span>{t('tab_refurb')}</span>
        </button>
    </div>
</div>

{/* Filter Bar: Brand Pills & Quick Search with 🏠 */}
<div className="flex flex-col lg:flex-row items-stretch lg:items-center justify-between gap-4">

    {/* Brand Filter Buttons */}
    <div className="flex items-center gap-1.5 overflow-x-auto pb-1 lg:pb-0 scrollbar-none">
        {BRANDS.map((brand) => (
            <button
                key={brand}
                onClick={() => setSelectedBrand(brand)}
                className={`px-3.5 py-1.5 rounded-lg text-xs font-semibold whitespace-nowrap transition border ${
                    selectedBrand === brand
                    ? 'bg-slate-900 text-white border-slate-900 shadow-xs'
                    : 'bg-white text-slate-700 border-slate-300 hover:bg-slate-50'
                }`}
            >
                {brand === 'All Brands' && language === 'hi' ? 'सभी ब्रांड' : brand}
            </button>
        ))}
    </div>

    {/* Search Input with 🏠 Shortcut */}
    <div className="relative min-w-[260px] sm:min-w-[320px]">
        <Search className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -translate-y-1/2 pointer-events-none"

        <input
            id="storefront-search-input"
            type="text"
            placeholder={buyingTab === 'new' ? t('search_new_placeholder') : t('search_refurb_placeholder')}
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            className="retail-input pl-9 pr-14 text-xs"
        />
        <span className="hidden sm:flex absolute right-2.5 top-1/2 -translate-y-1/2 items-center text-[10px] text
-slate-500 font-mono px-1.5 py-0.5 rounded bg-slate-100 border border-slate-300 pointer-events-none">
            🏠
        </span>
    </div>

```

```

    </div>
  </div>

  { /* Product Catalog Grid */}
  {loading ? (
    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
      {[1, 2, 3, 4, 5, 6].map((n) => (
        <div key={n} className="rounded-xl bg-white border border-slate-200 h-96 p-4 animate-pulse" />
      ))}
    </div>
  ) : filteredProducts.length === 0 ? (
    <div className="p-12 text-center rounded-xl bg-white border border-slate-200">
      <Smartphone className="w-10 h-10 text-slate-400 mx-auto mb-3" />
      <h4 className="text-base font-bold text-slate-900">{t('no_phones_found')}</h4>
      <p className="text-xs text-slate-500 mt-1 max-w-sm mx-auto">
        {language === 'hi'
          ? 'कृपया दूसरे ब्रांड पर क्लिक करें या सर्च बदलकर देखें।'
          : 'Try clearing your search query or selecting a different brand.'}
      </p>
      <button
        onClick={() => { setSelectedBrand('All Brands'); setSearchQuery(''); }}
        className="mt-4 btn-primary text-xs"
      >
        {t('reset_filters')}
      </button>
    </div>
  ) : (
    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
      {(filteredProducts || []).map((product) => (
        <ProductCard
          key={product?.id || Math.random()}
          product={product}
          onOpenCalculator={() => setSelectedForEmi(prod)}
        />
      ))}
    </div>
  )}

  { /* Live EMI Calculator Section */}
  <div id="live-emi-calculator" className="pt-4">
    <LiveEmiCalculatorWidget />
  </div>

  { /* EMI Partnership Badges */}
  <EMIBadges onOpenCalculator={() => setSelectedForEmi(prod)} />

  { /* Interactive EMI Calculator Modal */}
  {selectedForEmi && (
    <EmiCalculatorModal
      product={selectedForEmi}
      onClose={() => setSelectedForEmi(null)}
    />
  )}

</section>
);
}

```

```

import React, { useState } from 'react';
import {
  ShieldCheck,
  MessageCircle,
  ShoppingBag,
  Check,
  CreditCard,
  CheckCircle2,
  Sparkles
} from 'lucide-react';
import { useLanguage } from '../../../context/LanguageContext';
import { useCart } from '../../../context/CartContext';
import { SHOP_INFO } from '../../../data/mockData';

export default function ProductCard({ product, onOpenCalculator }) {
  const { t, language } = useLanguage();
  const { addToCart } = useCart();
  const [justAdded, setJustAdded] = useState(false);

  const isNew = product?.condition === 'new';
  const price = Number(product?.price) || 0;
  const originalPrice = Number(product?.original_price) || 0;
  const discount = originalPrice > price
    ? Math.round(((originalPrice - price) / originalPrice) * 100)
    : 0;

  const monthlyEmi = Math.round(price / 12);

  const handleAddToCart = () => {
    addToCart(product);
    setJustAdded(true);
    setTimeout(() => setJustAdded(false), 1800);
  };

  const handleWhatsAppBuy = () => {
    const title = product?.title || 'Smartphone';
    const condition = product?.condition || 'new';
    const text = isNew
      ? (language === 'hi'
        ? `नमस्ते Amit Mobile Shop, मुझे नया स्मार्टफोन *${title}* (₹${price.toLocaleString()}) खरीदना है। क्या यह आज उपलब्ध है? क्या 0% EMI ऑफर है?`
        : `Hello Amit Mobile Shop, I want to purchase *${title}* (Price: ₹${price.toLocaleString()}). Is it in stock at Khorare for same-day delivery / 0% EMI?`)
      : (language === 'hi'
        ? `नमस्ते Amit Mobile Shop, मुझे रिफर्बिश्ड फोन *${title}* (कंडीशन: ${condition}, ₹${price.toLocaleString()}) खरीदना है। कृपया बिल और वारंटी कन्फर्म करें।`
        : `Hello Amit Mobile Shop, I am interested in the certified pre-owned *${title}* (Condition: ${condition}, Price: ₹${price.toLocaleString()}). Please confirm shop warranty.`);

    const url = `https://wa.me/${SHOP_INFO.whatsapp}?text=${encodeURIComponent(text)}`;
    window.open(url, '_blank');
  };

  return (
    <div className="bg-white border border-slate-200 hover:border-slate-300 rounded-xl overflow-hidden shadow-sm hover:shadow-md transition-all duration-200 flex flex-col justify-between group">

      { /* Product Image Box */ }
      <div className="aspect-[4/3] bg-slate-50 border-b border-slate-100 p-4 flex items-center justify-center relative">
        <img
          src={product.image_url}
          alt={product.title}
          className="w-full h-full object-contain group-hover:scale-105 transition-transform duration-200"

```

```

        loading="lazy"
        onError={(e) => {
            e.target.src = "https://images.unsplash.com/photo-1511707171634-5f897ff02aa9?w=400&auto=format&fit=crop
&q=80";
        }}
    />

    {//* Condition Tag */}
    <div className="absolute top-3 left-3 flex flex-col gap-1">
        {isNew ? (
            <span className="px-2.5 py-0.5 rounded text-[10px] font-bold uppercase tracking-wider bg-slate-900 text
-white shadow-xs">
                Brand New
            </span>
        ) : (
            <span className="px-2.5 py-0.5 rounded text-[10px] font-bold uppercase tracking-wider bg-emerald-700 te
xt-white shadow-xs">
                Pre-Owned
            </span>
        )}

        {discount > 0 && (
            <span className="px-1.5 py-0.5 rounded text-[10px] font-bold bg-rose-600 text-white w-fit">
                {discount}% OFF
            </span>
        )}
    </div>

    {//* In Stock Pill */}
    <div className="absolute top-3 right-3 text-[11px] font-semibold text-emerald-700 bg-white/90 border border
-slate-200 px-2 py-0.5 rounded shadow-xs">
        In Stock
    </div>
</div>

{//* Product Information */}
<div className="p-4 flex-1 flex flex-col justify-between space-y-3">
    <div>
        {//* Brand & RAM/Storage */}
        <div className="flex items-center gap-1.5 text-xs text-slate-500 mb-1">
            <span className="font-bold text-slate-800">{product.brand}</span>
            <span>•</span>
            <span>{product.ram_storage} || 'Standard Specs'</span>
            <span>{product.color} && (
                <>
                    <span>•</span>
                    <span className="text-slate-500 truncate max-w-[100px]">{product.color}</span>
                </>
            )}
        </div>

        {//* Title */}
        <h3 className="text-base font-bold text-slate-900 line-clamp-1 font-['Poppins'] group-hover:text-primary-
800 transition-colors">
            {product.title}
        </h3>

        <p className="mt-1 text-xs text-slate-500 line-clamp-2 leading-relaxed">
            {product.description}
        </p>

        {//* Warranty Note */}
        <div className="mt-2.5 flex items-center gap-1.5 text-xs text-slate-600">
            <ShieldCheck className="w-3.5 h-3.5 text-emerald-700 shrink-0" />
            <span className="truncate">{product.warranty_info} || 'Official Warranty'</span>
        </div>

        {//* 0% EMI Badge Pill */}
        <div className="mt-2.5 py-1.5 px-2.5 rounded-lg bg-amber-50 border border-amber-200 flex items-center jus
tify-between text-xs">

```

```

<span className="inline-flex items-center gap-1 font-bold text-amber-900 bg-amber-200/80 px-2 py-0.5 rounded text-[11px]">
  <Sparkles className="w-3 h-3 text-amber-700" />
  <span>0% EMI</span>
</span>
<span className="font-bold text-slate-800">₹{monthlyEmi.toLocaleString()}/माह</span>
{onOpenCalculator && (
  <button
    type="button"
    onClick={() => onOpenCalculator(product)}
    className="text-[11px] text-blue-700 hover:underline font-bold"
  >
    कैलकुलेटर
  </button>
)}
</div>
</div>

{/* Pricing & CTAs */}
<div className="pt-3 border-t border-slate-100">
  <div className="flex items-baseline justify-between mb-3">
    <div>
      <span className="text-[10px] uppercase tracking-wider text-slate-400 font-medium block">
        {t('offer_price')}
      </span>
      <div className="flex items-baseline gap-2">
        <span className="text-xl font-black text-slate-950 font-['Poppins']">
          ₹{price.toLocaleString()}
        </span>
        {originalPrice > 0 && (
          <span className="text-xs text-slate-400 line-through">
            ₹{originalPrice.toLocaleString()}
          </span>
        )}
      </div>
    </div>
  </div>
</div>

{/* Two Clean Buttons */}
<div className="grid grid-cols-2 gap-2">
  <button
    onClick={handleAddToCart}
    className={`py-2.5 px-2.5 rounded-xl text-xs font-bold transition-all flex items-center justify-center gap-1.5`}
  >
    {justAdded
      ? 'bg-emerald-700 text-white'
      : 'bg-slate-900 hover:bg-slate-800 text-white shadow-xs'}
  </button>
  <button
    onClick={handleWhatsAppBuy}
    className="py-2.5 px-2.5 rounded-xl bg-emerald-600 hover:bg-emerald-700 text-white text-xs font-bold transition-all shadow-sm flex items-center justify-center gap-1.5 font-hindi"
    title="WhatsApp पर तुरंत बुक करें"
  >
    <MessageCircle className="w-3.5 h-3.5" />
    <span>बुक करें</span>
  </button>

```

```
        </button>
      </div>
    </div>

    </div>
  </div>
);
}
```

frontend/src/components/BuyingHub/EmiCalculatorModal.jsx - Interactive Down Payment & Tenure Calculator

```
frontend/src/components/BuyingHub/EmiCalculatorModal.jsx
```

JavaScript/JSX

```
import React, { useState } from 'react';
import { X, Calculator, MessageCircle, Info } from 'lucide-react';
import { useLanguage } from '../../context/LanguageContext';
import { SHOP_INFO } from '../../data/mockData';

export default function EmiCalculatorModal({ product, onClose }) {
  const [t, language] = useLanguage();
  if (!product) return null;

  const price = product.price || 25000;
  const [downPayment, setDownPayment] = useState(Math.round(price * 0.2));
  const [tenure, setTenure] = useState(6);
  const [partner, setPartner] = useState(product.emi_samsung ? "Samsung Finance+" : "Bajaj Finserv");

  const loanAmount = Math.max(0, price - downPayment);
  const monthlyEmi = Math.round(loanAmount / tenure);

  const handleWhatsAppInquiry = () => {
    const text = language === 'hi'
      ? `नमस्ते Amit Mobile Shop, मुझे "${product.title}" आसान किश्तों (EMI) पर लेना है।  
कीमत: ₹${price.toLocaleString()}  
डाउन पेमेंट: ₹${downPayment.toLocaleString()}  
किश्त अवधि: ${tenure} महीने (लगभग ₹${monthlyEmi.toLocaleString()}/महीना)  
पसंदीदा फाइनेंस: ${partner}  
कृपया जरूरी दस्तावेज व उपलब्धता कन्फर्म करें।`
      : `Hello Amit Mobile Shop, I want to purchase "${product.title}" on EMI.  
Price: ₹${price.toLocaleString()}  
Down Payment: ₹${downPayment.toLocaleString()}  
Tenure: ${tenure} Months (Approx ₹${monthlyEmi.toLocaleString()}/month)  
Financing: ${partner}  
Please confirm document requirements and availability.`;

    const url = `https://wa.me/${SHOP_INFO.whatsapp}?text=${encodeURIComponent(text)}`;
    window.open(url, '_blank');
  };

  return (


/* Close Button */



button



onClick={onClose}



className="absolute top-5 right-5 p-2 rounded-xl bg-slate-100 text-slate-500 hover:text-slate-900 hover:bg-slate-200 transition"



<X className="w-5 h-5" />



/* Modal Header */



div className="flex items-center gap-3 mb-5">



div className="p-3 rounded-2xl bg-primary-100 text-primary-700">



Calculator className="w-6 h-6" />



h3 className="text-xl font-black text-slate-900 font-['Poppins']">



{t('modal_calc_title')}



p className="text-xs text-slate-500 font-semibold truncate max-w-[280px]">



{product.title}


```

```

    </p>
  </div>
</div>

{/* Price & Monthly EMI Summary Box */}
<div className="p-4 rounded-2xl bg-primary-50 border-2 border-primary-200 mb-6">
  <div className="flex justify-between items-center text-xs text-slate-600 mb-1 font-bold">
    <span>{t('modal_device_price')}

```

```

<div className="grid grid-cols-3 gap-2">
  {[ 'Bajaj Finserv', 'TVS Credit', 'Samsung Finance+' ].map((p) => (
    <button
      key={p}
      type="button"
      onClick={() => setPartner(p)}
      className={`p-2 text-center rounded-xl text-xs font-bold transition border-2 ${
        partner === p
          ? 'bg-primary-50 text-primary-900 border-primary-600'
          : 'bg-white text-slate-600 border-slate-200 hover:bg-slate-50'
      }`}
    >
      {p}
    </button>
  ) )}
</div>

</div>

{/* Document Tip Notice */}
<div className="p-3.5 rounded-xl bg-gold-50 border border-gold-300 flex items-start gap-2.5 mb-6 text-xs text-gold-700 font-medium">
  <Info className="w-4 h-4 text-gold-700 shrink-0 mt-0.5" />
  <span>
    {t('modal_doc_tip')}
  </span>
</div>

{/* WhatsApp Apply Action Button */}
<button
  onClick={handleWhatsAppInquiry}
  className="w-full py-4 rounded-2xl bg-whatsapp-600 hover:bg-whatsapp-700 text-white font-black text-sm shadow-md transition flex items-center justify-center gap-2"
  >
  <MessageCircle className="w-4 h-4" />
  <span>{t('modal_apply_whatsapp')}</span>
</button>

</div>
</div>
);
}

```

```

import React, { useState } from 'react';
import { Wrench, Clock, ShieldCheck, Cpu, PhoneCall, MessageCircle, MapPin, Search } from 'lucide-react';
import RepairEstimator from './RepairEstimator';
import JobSheetTracker from './JobSheetTracker';
import { useLanguage } from '../../../context/LanguageContext';
import { SHOP_INFO } from '../../../data/mockData';

export default function RepairingHub() {
  const { t, language } = useLanguage();
  const [subTab, setSubTab] = useState('estimator'); // 'estimator' or 'tracker'

  return (
    <section className="space-y-8">

      {/* 3-Step Simple Repair Process Banner (Specially designed for rural/suburban junction audience) */}
      <div className="bg-gradient-to-br from-amber-500 via-amber-600 to-amber-700 rounded-3xl p-6 sm:p-8 text-white shadow-xl shadow-amber-600/20 border-b-[6px] border-amber-800">
        <div className="max-w-3xl mb-6">
          <div className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-white/20 backdrop-blur-xs text-xs font-bold uppercase tracking-wider text-amber-100 mb-2 border border-white/20">
            <Wrench className="w-3.5 h-3.5 text-amber-200" />
            <span>सरल व पारदर्शी 3-स्टेप रिपेयरिंग</span>
          </div>
          <h2 className="text-2xl sm:text-3xl lg:text-4xl font-black font-hindi tracking-tight">
            मोबाइल स्क्रीन, फोल्डर या बैटरी 1 घंटे में बदलवाएं
          </h2>
          <p className="mt-2 text-xs sm:text-sm text-amber-100 leading-relaxed">
            खोड़ारे चौराहे की विश्वसनीय शॉप – कोई छुपा हुआ चार्ज नहीं, टेस्टिंग के बाद असली बिल व दुकान की पक्की वारंटी।
          </p>
        </div>

        {/* 3 Step Cards */}
        <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">

          {/* Step 1 */}
          <div className="bg-white/10 backdrop-blur-md rounded-2xl p-4 border border-white/20 flex flex-col justify-between">
            <div className="flex items-center justify-between mb-3">
              <span className="w-8 h-8 rounded-xl bg-white text-amber-700 font-black text-sm flex items-center justify-center shadow-sm">
                1
              </span>
              <MapPin className="w-5 h-5 text-amber-200" />
            </div>
            <div>
              <h3 className="text-base font-bold font-hindi text-white">दुकान पर लाएं</h3>
              <p className="text-xs text-amber-100 mt-1">
                खोड़ारे चौराहा, कुक नगर ग्रिंट रोड पर अपना फोन लाएं या फोन पर समस्या बताएं।
              </p>
            </div>
          </div>

          {/* Step 2 */}
          <div className="bg-white/10 backdrop-blur-md rounded-2xl p-4 border border-white/20 flex flex-col justify-between">
            <div className="flex items-center justify-between mb-3">
              <span className="w-8 h-8 rounded-xl bg-white text-amber-700 font-black text-sm flex items-center justify-center shadow-sm">
                2
              </span>
              <Search className="w-5 h-5 text-amber-200" />
            </div>
            <div>
              <h3 className="text-base font-bold font-hindi text-white">तुरंत मुफ्त जांच व रेट</h3>
            </div>
          </div>

        </div>

    </section>
  );
}

```

```

        <p className="text-xs text-amber-100 mt-1">
            माइक्रोस्कोप व टेस्टिंग से सही फॉल्ट पहचानें और काम शुरू होने से पहले फिक्स रेट जानें।
        </p>
    </div>
</div>

{/* Step 3 */}
<div className="bg-white/10 backdrop-blur-md rounded-2xl p-4 border border-white/20 flex flex-col justify
-between">
    <div className="flex items-center justify-between mb-3">
        <span className="w-8 h-8 rounded-xl bg-white text-amber-700 font-black text-sm flex items-center just
ify-center shadow-sm">
            3
        </span>
        <ShieldCheck className="w-5 h-5 text-emerald-300" />
    </div>
    <div>
        <h3 className="text-base font-bold font-hindi text-white">1 घंटे में पक्की वारंटी</h3>
        <p className="text-xs text-amber-100 mt-1">
            ओरिजिनल फोल्डर व बैटरी फिटिंग, काउंटर पर पूरी टेस्टिंग और शॉप गारंटी रसीद।
        </p>
    </div>
</div>

{/* Direct Action Booking Buttons */}
<div className="flex flex-wrap items-center gap-3 pt-2">
    <a
        href={`tel:${SHOP_INFO.phone1}`}
        className="px-5 py-3 rounded-2xl bg-white text-amber-800 hover:bg-amber-50 font-bold text-xs sm:text-sm
transition-all duration-200 shadow-md flex items-center gap-2"
    >
        <PhoneCall className="w-4 h-4 text-amber-600" />
        <span>सीधे कॉल करें (+91 {SHOP_INFO.phone1})</span>
    </a>

    <a
        href={`https://wa.me/${SHOP_INFO.whatsapp}?text=${encodeURIComponent(
            'नमस्ते Amit Mobile Shop, मुझे अपना मोबाइल रिपेयर करवाना है (स्क्रीन / बैटरी / चार्जिंग)। कृपया रेट व समय बताएं।'
        )}`}
        target="_blank"
        rel="noopener noreferrer"
        className="px-5 py-3 rounded-2xl bg-emerald-600 hover:bg-emerald-700 text-white font-bold text-xs sm:te
xt-sm transition-all duration-200 shadow-md flex items-center gap-2"
    >
        <MessageCircle className="w-4 h-4" />
        <span>WhatsApp पर रिपेयर बुक करें</span>
    </a>

    <span className="text-xs text-amber-100 font-medium ml-auto hidden sm:inline">
        ⚡ स्क्रीन और बैटरी तुरंत बदलें
    </span>
</div>
</div>

{/* Repairing Hub Header & Sub-tabs */}
<div className="bg-white border border-slate-200 rounded-2xl p-5 sm:p-6 shadow-xs flex flex-col md:flex-row i
tems-start md:items-center justify-between gap-4">
    <div>
        <div className="flex items-center gap-2 mb-1">
            <Wrench className="w-5 h-5 text-amber-700" />
            <h2 className="text-xl sm:text-2xl font-black text-slate-900 font-hindi">
                रिपेयरिंग रेट कैलकुलेटर एवं लाइव स्टेटस ट्रैकिंग
            </h2>
        </div>
        <p className="text-xs sm:text-sm text-slate-500 font-normal">
            घर बैठे अपनी स्क्रीन, बैटरी या चार्जिंग का अनुमानित खर्च चेक करें या अपनी जॉब शीट (रसीद) ट्रैक करें।
        </p>
    </div>

```

```

    { /* Sub-Tabs: Estimator vs Job Sheet Tracker */ }
    <div className="inline-flex p-1 rounded-xl bg-slate-100 border border-slate-200 w-full md:w-auto">
      <button
        onClick={() => setSubTab('estimator')}
        className={`flex-1 md:flex-none flex items-center justify-center gap-2 px-5 py-2.5 rounded-lg font-bold
text-xs sm:text-sm transition-all ${
          subTab === 'estimator'
            ? 'bg-amber-600 text-white shadow-xs'
            : 'text-slate-600 hover:text-slate-900'
        }}`>
        <span>{t('subtab_estimator')} (खर्च जानें)</span>
      </button>

      <button
        onClick={() => setSubTab('tracker')}
        className={`flex-1 md:flex-none flex items-center justify-center gap-2 px-5 py-2.5 rounded-lg font-bold
text-xs sm:text-sm transition-all ${
          subTab === 'tracker'
            ? 'bg-amber-600 text-white shadow-xs'
            : 'text-slate-600 hover:text-slate-900'
        }}`>
        <Clock className="w-3.5 h-3.5" />
        <span>{t('subtab_tracker')} (स्थिति जाँचें)</span>
      </button>
    </div>
  </div>

  { /* 3 Grounded Service Guarantees */ }
  <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
    <div className="p-4 rounded-xl bg-white border border-slate-200 flex items-center gap-3.5 shadow-xs">
      <div className="p-2 rounded-lg bg-amber-50 text-amber-800 border border-amber-200 shrink-0">
        <Clock className="w-5 h-5" />
      </div>
      <div>
        <h4 className="text-xs sm:text-sm font-bold text-slate-900">
          {t('express_guarantee_1')}
        </h4>
        <p className="text-[11px] text-slate-500 font-normal">Express 1-hour screen & battery fitting</p>
      </div>
    </div>

    <div className="p-4 rounded-xl bg-white border border-slate-200 flex items-center gap-3.5 shadow-xs">
      <div className="p-2 rounded-lg bg-blue-50 text-blue-800 border border-blue-200 shrink-0">
        <Cpu className="w-5 h-5" />
      </div>
      <div>
        <h4 className="text-xs sm:text-sm font-bold text-slate-900">
          {t('express_guarantee_2')}
        </h4>
        <p className="text-[11px] text-slate-500 font-normal">Microscope diagnosis for charging & IC issues</p>
      </div>
    </div>

    <div className="p-4 rounded-xl bg-white border border-slate-200 flex items-center gap-3.5 shadow-xs">
      <div className="p-2 rounded-lg bg-emerald-50 text-emerald-800 border border-emerald-200 shrink-0">
        <ShieldCheck className="w-5 h-5" />
      </div>
      <div>
        <h4 className="text-xs sm:text-sm font-bold text-slate-900">
          {t('express_guarantee_3')}
        </h4>
        <p className="text-[11px] text-slate-500 font-normal">Tested on counter with transparent printed receipt</p>
      </div>
    </div>
  </div>

```

```
    { /* Sub-tab Views */  
    <div className="bg-white border border-slate-200 rounded-xl p-6 shadow-xs">  
      {subTab === 'estimator' ? (  
        <RepairEstimator />  
      ) : (  
        <JobSheetTracker />  
      )}  
    </div>  
  
  </section>  
);  
}
```

frontend/src/components/RepairingHub/JobSheetTracker.jsx

JavaScript/JSX

```

import React, { useState } from 'react';
import { Search, CheckCircle2, Clock, Wrench, PackageCheck, AlertCircle, MessageCircle, FileText, Smartphone } from 'lucide-react';
import { fetchRepairStatus } from '../../services/api';
import { useLanguage } from '../../context/LanguageContext';
import { SHOP_INFO } from '../../data/mockData';

export default function JobSheetTracker() {
  const { t, language } = useLanguage();
  const [jobId, setJobId] = useState('AMS-101');
  const [loading, setLoading] = useState(false);
  const [jobData, setJobData] = useState(null);
  const [error, setError] = useState(null);

  const TRACKING_STAGES = [
    { id: 'Received', label: t('stage_received'), desc: t('stage_received_desc'), icon: FileText },
    { id: 'In Repair', label: t('stage_in_repair'), desc: t('stage_in_repair_desc'), icon: Wrench },
    { id: 'Ready for Pickup', label: t('stage_ready'), desc: t('stage_ready_desc'), icon: PackageCheck },
    { id: 'Delivered', label: t('stage_delivered'), desc: t('stage_delivered_desc'), icon: CheckCircle2 },
  ];

  const handleTrack = async (idToTrack = null) => {
    const targetId = (idToTrack || jobId).trim();
    if (!targetId) {
      setError(language === 'hi' ? 'कृपया सही रसीद / जॉब शीट नंबर दर्ज करें' : 'Please enter a valid Job Sheet ID');
      return;
    }
    setError(null);
    setLoading(true);

    try {
      const data = await fetchRepairStatus(targetId);
      setJobData(data);
    } catch (err) {
      setJobData(null);
      setError(err.message || 'Job Sheet ID not found.');
```

```

    } finally {
      setLoading(false);
    }
  };

  const getStageIndex = (status) => {
    if (!status) return 0;
    const s = status.toLowerCase();
    if (s.includes('deliver')) return 3;
    if (s.includes('ready') || s.includes('pickup') || s.includes('quality')) return 2;
    if (s.includes('repair') || s.includes('parts') || s.includes('diagno')) return 1;
    return 0;
  };

  const currentStageIdx = jobData ? getStageIndex(jobData.status) : 0;

  const handleWhatsAppStatusInquiry = () => {
    if (!jobData) return;
    const text = language === 'hi'
      ? `नमस्ते Amit Mobile Shop, मेरी रिपेयरिंग रसीद नंबर *${jobData.job_sheet_id}* (${jobData.device_brand} ${jobData.device_model}) का स्टेटस जानना है। क्या फोन तैयार हो गया है?`
      : `Hello Amit Mobile Shop, I want an update on my repair Job Sheet *${jobData.job_sheet_id}* (${jobData.device_brand} ${jobData.device_model}). Status is "${jobData.status}". When can I collect it?`;

    const url = `https://wa.me/${SHOP_INFO.whatsapp}?text=${encodeURIComponent(text)}`;
    window.open(url, '_blank');
  };
};

```

```

return (
  <div className="rounded-3xl bg-white border-2 border-slate-200 p-6 sm:p-8 shadow-sm">

    {/* Header */}
    <div className="flex flex-col sm:flex-row items-start sm:items-center justify-between gap-4 mb-6 pb-4 border-b border-slate-200">
      <div>
        <div className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-whatsapp-50 text-whatsapp-700 text-xs font-bold mb-2">
          <Clock className="w-3.5 h-3.5" />
          <span>लाइव रसीद ट्रैकिंग (Receipt Tracking)</span>
        </div>
        <h3 className="text-xl sm:text-2xl font-black text-slate-900 font-['Poppins']">
          {t('tracker_heading')}
        </h3>
        <p className="text-xs sm:text-sm text-slate-600 mt-1 font-medium">
          {t('tracker_subheading')}
        </p>
      </div>
    </div>

    {/* Search Input & Demo Chips */}
    <div className="space-y-4">
      <div className="flex flex-col sm:flex-row gap-2.5">
        <div className="relative flex-1">
          <Search className="w-4 h-4 text-slate-400 absolute left-4 top-1/2 -translate-y-1/2" />
          <input
            type="text"
            value={jobId}
            onChange={(e) => setJobId(e.target.value.toUpperCase())}
            placeholder={t('tracker_input_placeholder')}
            className="w-full pl-11 pr-4 py-3.5 rounded-2xl retail-input text-sm font-mono uppercase font-black"
          />
        </div>
        <button
          onClick={() => handleTrack()}
          disabled={loading}
          className="px-7 py-3.5 rounded-2xl bg-primary-700 hover:bg-primary-800 text-white font-black text-sm shadow-md transition flex items-center justify-center gap-2 disabled:opacity-50 whitespace-nowrap"
        >
          {loading ? (language === 'hi' ? 'चेक कर रहे हैं...' : 'Checking...') : t('tracker_btn')}
        </button>
      </div>

      {/* Quick Demo Chips */}
      <div className="flex flex-wrap items-center gap-2 text-xs">
        <span className="text-slate-500 font-bold">{t('quick_test_chips')}</span>
        {[
          'AMS-101', 'AMS-102', 'AMS-103', 'AMS-104'
        ].map((demoId) => (
          <button
            key={demoId}
            type="button"
            onClick={() => {
              setJobId(demoId);
              handleTrack(demoId);
            }}
            className="px-3 py-1.5 rounded-xl bg-slate-100 hover:bg-slate-200 border border-slate-300 text-primary-800 text-xs font-mono font-bold transition"
          >
            {demoId}
          </button>
        ))}
      </div>

      {error && (
        <div className="mt-6 p-4 rounded-2xl bg-rose-50 border border-rose-300 text-rose-700 text-xs flex items-center gap-2.5 font-medium">
          <AlertCircle className="w-5 h-5 shrink-0" />
          <span>{error}</span>
        </div>
      )}
    </div>
  </div>
)

```

```

    })

    { /* Tracking Result Banner & Timeline */ }
    { jobData && (
      <div className="mt-8 space-y-6 animate-in fade-in duration-200">

        { /* Status Header Banner */ }
        <div className="p-5 rounded-2xl bg-slate-50 border-2 border-slate-200 flex flex-col sm:flex-row items-start sm:items-center justify-between gap-4">
          <div>
            <div className="flex items-center gap-2">
              <span className="px-3 py-1 rounded-md bg-primary-700 text-white font-mono text-xs font-black">
                रसीद: {jobData.job_sheet_id}
              </span>
              <span className="text-xs text-slate-500 font-medium">
                दिनांक: {jobData.created_at || 'आज'}
              </span>
            </div>
            <h4 className="text-lg font-black text-slate-900 mt-1.5 flex items-center gap-2 font-['Poppins']">
              <Smartphone className="w-5 h-5 text-primary-600" />
              <span>{jobData.device_brand} {jobData.device_model}</span>
            </h4>
          </div>

          <div className="flex items-center gap-2">
            <span className="text-xs text-slate-500 font-bold">वर्तमान स्थिति:</span>
            <span className={`px-4 py-1.5 rounded-full text-xs font-black uppercase tracking-wider ${
              jobData.status === 'Ready for Pickup'
                ? 'bg-whatsapp-50 text-whatsapp-700 border-2 border-whatsapp-300'
                : jobData.status === 'In Repair'
                ? 'bg-gold-50 text-gold-700 border-2 border-gold-300'
                : 'bg-primary-50 text-primary-800 border-2 border-primary-300'
            }`}>
              {jobData.status}
            </span>
          </div>
        </div>

        { /* Stepped Progress Timeline */ }
        <div className="p-6 rounded-2xl bg-slate-50 border border-slate-200">
          <div className="relative">

            { /* Progress Line */ }
            <div className="hidden sm:block absolute top-5 left-8 right-8 h-1 bg-slate-200">
              <div
                className="h-full bg-primary-600 transition-all duration-300"
                style={{ width: `${(currentStageIdx / (TRACKING_STAGES.length - 1)) * 100}%` }}
              />
            </div>

            { /* Steps */ }
            <div className="grid grid-cols-1 sm:grid-cols-4 gap-4 relative">
              {TRACKING_STAGES.map((stage, idx) => {
                const Icon = stage.icon;
                const isCompleted = idx <= currentStageIdx;
                const isCurrent = idx === currentStageIdx;

                return (
                  <div key={stage.id} className="flex sm:flex-col items-center sm:text-center gap-3">
                    <div className={`w-10 h-10 rounded-2xl flex items-center justify-center shrink-0 z-10 transition-all duration-200 ${
                      isCurrent
                        ? 'bg-primary-700 text-white ring-4 ring-primary-100 shadow-md'
                        : isCompleted
                        ? 'bg-whatsapp-600 text-white'
                        : 'bg-white border-2 border-slate-300 text-slate-400'
                    }`}>
                      <Icon className="w-5 h-5" />
                    </div>
                    <div>

```

```

        <span className={`text-xs font-black block ${
            isCurrent ? 'text-primary-800' : isCompleted ? 'text-slate-900' : 'text-slate-400'
        }`}>
            {stage.label}
        </span>
        <span className="text-[11px] text-slate-500 hidden sm:block font-medium">
            {stage.desc}
        </span>
    </div>
</div>
);
}}}
</div>
</div>
</div>

{/* Detailed Diagnosis & Notes Card */}
<div className="p-5 rounded-2xl bg-white border border-slate-200 space-y-4 text-xs">
    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4 font-medium">
        <div>
            <span className="text-slate-500 block font-bold">{t('cust_name_label')}</span>
            <span className="font-bold text-slate-900 text-sm">{jobData.customer_name}</span>
        </div>
        <div>
            <span className="text-slate-500 block font-bold">{t('issue_label')}</span>
            <span className="font-bold text-slate-900 text-sm">{jobData.issue_type}</span>
        </div>
        <div>
            <span className="text-slate-500 block font-bold">{t('est_bill_label')}</span>
            <span className="font-black text-primary-700 font-mono text-base">
                ₹{jobData.estimated_cost?.toLocaleString()}
            </span>
        </div>
        <div>
            <span className="text-slate-500 block font-bold">{t('handled_by_label')}</span>
            <span className="font-bold text-slate-900">
                {jobData.technician_name || 'Amit Mobile Shop Expert'}
            </span>
        </div>
    </div>

    {/* Technician Note */}
    {jobData.technician_notes && (
        <div className="pt-3 border-t border-slate-200">
            <span className="text-slate-600 font-bold block mb-1">
                {t('tech_note_label')}
            </span>
            <p className="p-3.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-800 leading-relaxed font-semibold">
                {jobData.technician_notes}
            </p>
        </div>
    )}
</div>

{/* WhatsApp Status Chat Button */}
<button
    onClick={handleWhatsAppStatusInquiry}
    className="w-full py-4 rounded-2xl bg-whatsapp-600 hover:bg-whatsapp-700 text-white font-black text-xs sm:text-sm shadow-md transition flex items-center justify-center gap-2">
    >
    <MessageCircle className="w-4 h-4" />
    <span>{t('track_wa_inquiry')}</span>
</button>

</div>
)}
</div>

```

```
);  
}
```

frontend/src/components/RepairingHub/RepairEstimator.jsx

JavaScript/JSX

```

import React, { useState } from 'react';
import { Wrench, Clock, ShieldCheck, CheckCircle, ChevronDown, ChevronUp, MessageCircle, AlertCircle } from 'lucide-react';
import { calculateEstimate } from '../../services/api';
import { useLanguage } from '../../context/LanguageContext';
import { SHOP_INFO } from '../../data/mockData';

const COMMON_BRANDS = [
  'Samsung', 'Apple', 'Vivo', 'Realme', 'Xiaomi / Redmi', 'OnePlus', 'Oppo'
];

export default function RepairEstimator() {
  const { t, language } = useLanguage();
  const [brand, setBrand] = useState('Samsung');
  const [model, setModel] = useState('Galaxy M31');
  const [issue, setIssue] = useState('Display / Touch Screen Replacement');
  const [notes, setNotes] = useState('');
  const [loading, setLoading] = useState(false);
  const [estimate, setEstimate] = useState(null);
  const [showBreakdown, setShowBreakdown] = useState(false);
  const [error, setError] = useState(null);

  const COMMON_ISSUES = [
    {
      label: language === 'hi' ? 'स्क्रीन / कॉम्बो फूटा हुआ' : 'Broken Screen / Display Combo',
      issue: 'Display / Touch Screen Replacement'
    },
    {
      label: language === 'hi' ? 'बैटरी बैकअप कम / जल्दी खत्म' : 'Battery Draining Fast / Low Backup',
      issue: 'Battery Replacement & Health Check'
    },
    {
      label: language === 'hi' ? 'चार्जिंग पिन / सॉकेट ढीला' : 'Charging Port / Pin Loose',
      issue: 'Type-C / Lightning Port Replacement'
    },
    {
      label: language === 'hi' ? 'कैमरा धुंधला / ग्लास टूटा' : 'Camera Blurry / Broken Glass',
      issue: 'Camera Lens / Sensor Replacement'
    },
    {
      label: language === 'hi' ? 'पानी में गिरा फोन / लिक्विड' : 'Water Dropped / Liquid Damage',
      issue: 'Water Damage Ultrasonic Service'
    },
    {
      label: language === 'hi' ? 'स्पीकर / माइक में आवाज नहीं' : 'Speaker / Mic Not Working',
      issue: 'Speaker / Earpiece / Mic Replacement'
    },
    {
      label: language === 'hi' ? 'फोन ऑन नहीं हो रहा / मदरबोर्ड' : 'Dead Phone / Motherboard Issue',
      issue: 'Chip-level Motherboard Diagnostic'
    }
  ];

  const handleEstimate = async (e) => {
    if (e) e.preventDefault();
    if (!model.trim()) {
      setError(language === 'hi' ? 'कृपया मोबाइल का मॉडल नाम लिखें' : 'Please enter your phone model');
      return;
    }
    setError(null);
    setLoading(true);

    try {
      const res = await calculateEstimate(brand, model, issue, notes);
    }
  }

```

```

    setEstimate(res);
  } catch (err) {
    console.error(err);
    setError(language === 'hi' ? 'अनुमान लगाने में समस्या आई। कृपया व्हाट्सएप पर संपर्क करें।' : 'Could not calculate estimate. Please message on WhatsApp.');
```

ate. Please message on WhatsApp.');

```

    } finally {
      setLoading(false);
    }
  }
};

return (
  <div className="rounded-3xl bg-white border-2 border-slate-200 p-6 sm:p-8 shadow-sm">

    {/* Header */}
    <div className="flex flex-col sm:flex-row items-start sm:items-center justify-between gap-4 mb-6 pb-4 border-b border-slate-200">
      <div>
        <div className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-primary-50 text-primary-800 text-xs font-bold mb-2">
          <Wrench className="w-3.5 h-3.5" />
          <span>{language === 'hi' ? 'दुकान का प्रमाणित रेट चार्ट' : 'Standard Store Price Guide'}</span>
        </div>
        <h3 className="text-xl sm:text-2xl font-black text-slate-900 font-['Poppins']">
          {language === 'hi' ? 'मोबाइल रिपेयरिंग खर्च और समय का अनुमान' : 'Mobile Repair Cost & Turnaround Estimator'}
        </h3>
        <p className="text-xs sm:text-sm text-slate-600 mt-1 font-medium">
          {language === 'hi' ? 'दुकान आने से पहले घर बैठे सही खर्च का पता लगाएं।' : 'Get transparent cost estimates before visiting Amit Mobile Shop.'}
        </p>
      </div>

      <div className="flex items-center gap-2 text-xs text-slate-700 bg-slate-100 px-4 py-2.5 rounded-xl border border-slate-200 font-medium">
        <Clock className="w-4 h-4 text-whatsapp-600" />
        <span>{language === 'hi' ? 'औसत रिपेयर समय:' : 'Avg Repair Time:'} <strong>45 मिनट</strong></span>
      </div>
    </div>

    {/* Estimator Input Form */}
    <form onSubmit={handleEstimate} className="space-y-6">

      {/* Brand Selector */}
      <div>
        <label className="block text-xs font-black text-slate-800 uppercase tracking-wider mb-2.5">
          {t('step_1_brand')}
        </label>
        <div className="flex flex-wrap gap-2">
          {COMMON_BRANDS.map((b) => (
            <button
              key={b}
              type="button"
              onClick={() => setBrand(b)}
              className={`px-4 py-2.5 rounded-xl text-xs font-bold transition border-2 ${
                brand === b
                  ? 'bg-primary-700 text-white border-primary-700 shadow-sm'
                  : 'bg-slate-50 text-slate-700 border-slate-200 hover:bg-slate-100'
              }`}
            >
              {b}
            </button>
          ))}
        </div>
      </div>

      {/* Model Input */}
      <div>
        <label className="block text-xs font-black text-slate-800 uppercase tracking-wider mb-2">
          {t('step_2_model')}
        </label>

```

```

        <input
          type="text"
          value={model}
          onChange={(e) => setModel(e.target.value)}
          placeholder="उदा. Galaxy M31, iPhone 12, Redmi Note 11, Narzo 50..."
          className="w-full px-4 py-3 rounded-xl retail-input text-sm font-semibold"
          required
        />
      </div>

      { /* Issue Selector Chips */ }
      <div>
        <label className="block text-xs font-black text-slate-800 uppercase tracking-wider mb-2.5">
          {t('step_3_issue')}
        </label>
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-2.5">
          {COMMON_ISSUES.map((item) => (
            <button
              key={item.label}
              type="button"
              onClick={() => setIssue(item.issue)}
              className={`p-3.5 rounded-xl text-left text-xs font-bold transition border-2 flex items-center justify-between $
                {
                  issue === item.issue
                    ? 'bg-primary-50 text-primary-900 border-primary-600 shadow-sm'
                    : 'bg-white text-slate-700 border-slate-200 hover:bg-slate-50'
                }
              `}
            >
              <span>{item.label}</span>
              {issue === item.issue && <CheckCircle className="w-4 h-4 text-primary-700 shrink-0" />}
            </button>
          ))}
        </div>
      </div>

      { /* Optional Notes */ }
      <div>
        <label className="block text-xs font-bold text-slate-600 uppercase tracking-wider mb-2">
          {t('step_4_notes')}
        </label>
        <input
          type="text"
          value={notes}
          onChange={(e) => setNotes(e.target.value)}
          placeholder={language === 'hi' ? "उदा. टच काम नहीं कर रहा, हाथ से गिर गया था..." : "e.g. touch not responding, fell on floor..."}
          className="w-full px-4 py-2.5 rounded-xl retail-input text-xs font-medium"
        />
      </div>

      {error && (
        <div className="p-3.5 rounded-xl bg-rose-50 border border-rose-300 text-rose-700 text-xs flex items-center gap-2 font-medium">
          <AlertCircle className="w-4 h-4 shrink-0" />
          <span>{error}</span>
        </div>
      )}

      { /* Submit button */ }
      <button
        type="submit"
        disabled={loading}
        className="w-full py-4 rounded-2xl bg-primary-700 hover:bg-primary-800 text-white font-black text-sm sm:text-base shadow-md transition flex items-center justify-center gap-2 disabled:opacity-50"
      >
        {loading ? (
          <>
            <div className="w-5 h-5 border-2 border-white/30 border-t-white rounded-full animate-spin" />
            <span>{t('calculating')}</span>
          </>
        ) : (

```

```

    ) : (
      <>
        <Wrench className="w-5 h-5 text-gold-400" />
        <span>{t('calc_estimate_btn')}</span>
      </>
    )}
  </button>
</form>

{/* Estimation Results Card */}
{estimate && (
  <div className="mt-8 p-6 rounded-2xl bg-slate-50 border-2 border-primary-200 animate-in fade-in duration-200">

    <div className="flex flex-col sm:flex-row items-start sm:items-center justify-between gap-4 pb-4 border-b border-slate-200">
      <div>
        <span className="text-xs font-black text-primary-800 uppercase tracking-wider">
          {estimate.brand} • {estimate.model}
        </span>
        <h4 className="text-lg font-black text-slate-900 mt-1 font-['Poppins']">
          {estimate.issue}
        </h4>
      </div>

      <div className="text-left sm:text-right">
        <span className="text-xs text-slate-500 font-bold block">{t('est_cost_label')}</span>
        <span className="text-2xl sm:text-3xl font-black text-primary-700 font-['Poppins']">
          ₹{Math.round(estimate.estimated_min_cost).toLocaleString()} - ₹{Math.round(estimate.estimated_max_cost).toLocaleString()}
        </span>
      </div>
    </div>

    <div>
      <div>
        <div className="text-left sm:text-right">
          <span className="text-xs text-slate-500 font-bold block">{t('est_time_label')}</span>
          <span className="text-sm font-black text-slate-900 flex items-center gap-1.5 mt-0.5">
            <Clock className="w-4 h-4 text-primary-600" />
            {estimate.turnaround_time}
          </span>
        </div>
      </div>

      <div>
        <div className="text-left sm:text-right">
          <span className="text-xs text-slate-500 font-bold block">{t('est_quality_label')}</span>
          <span className="text-sm font-black text-slate-900 flex items-center gap-1.5 mt-0.5">
            <CheckCircle className="w-4 h-4 text-whatsapp-600" />
            {t('tested_grade')}
          </span>
        </div>
      </div>

      <div>
        <div className="text-left sm:text-right">
          <span className="text-xs text-slate-500 font-bold block">{t('est_warranty_label')}</span>
          <span className="text-sm font-black text-slate-900 flex items-center gap-1.5 mt-0.5">
            <ShieldCheck className="w-4 h-4 text-gold-600" />
            3 से 6 महीने
          </span>
        </div>
      </div>
    </div>

    <div>
      <p className="mt-4 text-xs text-slate-700 leading-relaxed font-medium">
        {estimate.summary}
      </p>

      <div className="mt-3 p-3.5 rounded-xl bg-gold-50 border border-gold-300 text-xs text-gold-900 flex items-start gap-2">
        <span className="font-bold shrink-0">{t('tech_tip_label')}</span>
        <span className="font-medium">{estimate.technician_tip}</span>
      </div>
    </div>
  </div>
)
}

```

```

</div>

{/* Cost Breakdown Toggle */}
<div className="mt-4 pt-3 border-t border-slate-200">
  <button
    type="button"
    onClick={() => setShowBreakdown(!showBreakdown)}
    className="text-xs font-bold text-primary-700 hover:text-primary-800 flex items-center gap-1"
  >
    <span>{showBreakdown ? t('hide_breakdown') : t('show_breakdown')}</span>
    {showBreakdown ? <ChevronUp className="w-4 h-4" /> : <ChevronDown className="w-4 h-4" />}
  </button>

  {showBreakdown && estimate.breakdown && (
    <div className="mt-3 p-3.5 rounded-xl bg-white border border-slate-200 text-xs space-y-2 font-medium">

      <div className="flex justify-between text-slate-600">
        <span>{t('part_cost_label')}</span>
        <span className="text-slate-900 font-bold">
          ₹{estimate.breakdown.part_cost_min} - ₹{estimate.breakdown.part_cost_max}
        </span>
      </div>

      <div className="flex justify-between text-slate-600">
        <span>{t('labor_cost_label')}</span>
        <span className="text-slate-900 font-bold">₹{estimate.breakdown.service_charge}</span>
      </div>

      <div className="flex justify-between text-slate-600">
        <span>दुकान वारंटी:</span>
        <span className="text-whatsapp-700 font-bold">{estimate.breakdown.warranty_provided}</span>
      </div>
    </div>
  )}
</div>

{/* Direct WhatsApp Booking Button */}
<div className="mt-5">
  <a
    href={estimate.whatsapp_link}
    target="_blank"
    rel="noopener noreferrer"
    className="w-full py-4 rounded-2xl bg-whatsapp-600 hover:bg-whatsapp-700 text-white font-black text-sm shadow-md transition flex items-center justify-center gap-2"
  >
    <MessageCircle className="w-4 h-4" />
    <span>{t('book_repair_wa')}</span>
  </a>
</div>

</div>
)}
</div>
);
}

```

frontend/src/components/Admin/AdminLoginPage.jsx

JavaScript/JSX

```

import React, { useState, useEffect } from 'react';
import { useNavigate, useLocation, Link } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import {
  ShieldCheck,
  Lock,
  User,
  Eye,
  EyeOff,
  ArrowLeft,
  AlertCircle,
  Loader2,
  Smartphone
} from 'lucide-react';
import './adminTheme.css';

export default function AdminLoginPage() {
  const [username, setUsername] = useState('admin');
  const [password, setPassword] = useState('admin123');
  const [showPassword, setShowPassword] = useState(false);
  const [localError, setLocalError] = useState('');

  const { login, isAuthenticated, isLoading, authError, clearError } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  // Redirect if already logged in
  useEffect(() => {
    if (isAuthenticated) {
      const destination = location.state?.from?.pathname || '/admin/dashboard';
      navigate(destination, { replace: true });
    }
  }, [isAuthenticated, navigate, location]);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLocalError('');
    clearError();

    if (!username.trim() || !password) {
      setLocalError('Please enter both username and password.');
      return;
    }

    const res = await login(username.trim(), password);
    if (res.success) {
      const destination = location.state?.from?.pathname || '/admin/dashboard';
      navigate(destination, { replace: true });
    }
  };

  const displayedError = localError || authError;

  return (
    <div className="min-h-screen bg-slate-950 text-white flex flex-col justify-between font-sans selection:bg-prima
ry-700 selection:text-white">

      { /* Top Header */ }
      <header className="w-full max-w-5xl mx-auto px-4 sm:px-6 py-5 flex items-center justify-between">
        <Link
          to="/"
          className="inline-flex items-center gap-2 text-xs font-semibold text-slate-400 hover:text-white transitio
n px-3 py-1.5 rounded-lg bg-slate-900 border border-slate-800"
        >

```

```

        <ArrowLeft className="w-4 h-4" />
        <span>Back to Storefront</span>
    </Link>

    <div className="flex items-center gap-2 text-xs text-slate-400">
        <span className="w-2 h-2 rounded-full bg-emerald-500" />
        <span>Authorized Personnel Only</span>
    </div>
</header>

{/* Main Login Card */}
<main className="flex-1 flex items-center justify-center px-4 py-8">
    <div className="w-full max-w-sm bg-slate-900 border border-slate-800 rounded-xl p-6 sm:p-8 shadow-lg">

        {/* Shop Header */}
        <div className="text-center mb-6">
            <div className="inline-flex items-center justify-center w-12 h-12 rounded-lg bg-primary-800 text-white shadow-xs mb-3">
                <Smartphone className="w-6 h-6" />
            </div>

            <h1 className="text-xl font-bold tracking-tight text-white font-['Poppins']">
                Amit Mobile Shop
            </h1>
            <p className="text-xs text-slate-400 mt-1">
                Store Owner & Admin Portal
            </p>
        </div>

        {/* Error Alert Message */}
        {displayedError && (
            <div className="mb-4 p-3 rounded-lg bg-rose-950/50 border border-rose-800/60 text-rose-300 text-xs flex items-start gap-2">
                <AlertCircle className="w-4 h-4 shrink-0 mt-0.5 text-rose-400" />
                <div className="flex-1 font-medium leading-relaxed">
                    {displayedError}
                </div>
            </div>
        )}

        {/* Form */}
        <form onSubmit={handleSubmit} className="space-y-4" noValidate>
            <div>
                <label>
                    htmlFor="admin-username"
                    className="block text-xs font-semibold text-slate-300 mb-1"
                >
                    Username
                </label>
                <div className="relative">
                    <User className="w-4 h-4 text-slate-500 absolute left-3 top-1/2 -translate-y-1/2" />
                    <input
                        id="admin-username"
                        type="text"
                        autoComplete="username"
                        required
                        value={username}
                        onChange={(e) => {
                            setUsername(e.target.value);
                            if (displayedError) clearError();
                        }}
                        placeholder="admin"
                        className="w-full bg-slate-950 border border-slate-700 rounded-lg pl-9 pr-3 py-2 text-xs text-white placeholder-slate-500 focus:outline-none focus:border-slate-500"
                    />
                </div>
            </div>

            <div>
                <label>

```

```

        htmlFor="admin-password"
        className="block text-xs font-semibold text-slate-300 mb-1"
    >
        Password
    </label>
    <div className="relative">
        <Lock className="w-4 h-4 text-slate-500 absolute left-3 top-1/2 -translate-y-1/2" />
        <input
            id="admin-password"
            type={showPassword ? 'text' : 'password'}
            autoComplete="current-password"
            required
            value={password}
            onChange={(e) => {
                setPassword(e.target.value);
                if (displayedError) clearError();
            }}
            placeholder="....."
            className="w-full bg-slate-950 border border-slate-700 rounded-lg pl-9 pr-10 py-2 text-xs text-wh
ite placeholder-slate-500 focus:outline-none focus:border-slate-500 font-mono"
        />
        <button
            type="button"
            onClick={() => setShowPassword(!showPassword)}
            className="absolute inset-y-0 right-0 pr-3 flex items-center text-slate-500 hover:text-slate-300"
        >
            {showPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
        </button>
    </div>
</div>

<button
    type="submit"
    disabled={isLoading}
    className="w-full mt-2 py-2.5 px-4 rounded-lg bg-primary-700 hover:bg-primary-600 text-white font-sem
ibold text-xs transition shadow-xs flex items-center justify-center gap-2"
>
    {isLoading ? (
        <>
            <Loader2 className="w-4 h-4 animate-spin" />
            <span>Signing In...</span>
        </>
    ) : (
        <>
            <ShieldCheck className="w-4 h-4" />
            <span>Sign In</span>
        </>
    )}
</button>
</form>

{/* Credentials Hint */}
<div className="mt-5 pt-3 border-t border-slate-800 text-center">
    <p className="text-[11px] text-slate-500">
        Default Login: <strong className="text-slate-300">admin</strong> / <strong className="text-slate-30
0">admin123</strong>
    </p>
</div>
</div>
</main>

{/* Footer */}
<footer className="w-full max-w-5xl mx-auto px-4 py-4 text-center text-xs text-slate-600">
    © Amit Mobile Shop • Khorare, Uttar Pradesh 271312
</footer>
</div>
);
}

```

frontend/src/components/Admin/AdminDashboard.jsx

JavaScript/JSX

```

import React, { useState, useEffect } from 'react';
import { useAuth } from '../../../context/AuthContext';
import { fetchAdminStats, fetchProducts, createProduct, updateProduct, deleteProduct } from '../../../services/api';
import { INITIAL_PRODUCTS, MOCK_REPAIRS } from '../../../data/mockData';
import AdminLayout from './AdminLayout';
import DashboardOverview from './DashboardOverview';
import ProductsManager from './ProductsManager';
import RepairsManager from './RepairsManager';
import InventoryManager from './InventoryManager';
import EmiManager from './EmiManager';
import SettingsManager from './SettingsManager';
import WhatsAppInquiriesManager from './WhatsAppInquiriesManager';
import SalesAnalyticsManager from './SalesAnalyticsManager';

export default function AdminDashboard() {
  const { admin, logout } = useAuth();
  const [activeTab, setActiveTab] = useState('dashboard');
  const [searchQuery, setSearchQuery] = useState('');

  // Live Stats
  const [stats, setStats] = useState({
    total_products: 0,
    in_stock_products: 0,
    total_repairs: 0,
    pending_repairs: 0
  });
  const [loadingStats, setLoadingStats] = useState(true);

  // Products Data
  const [products, setProducts] = useState(INITIAL_PRODUCTS);
  const [loadingProducts, setLoadingProducts] = useState(false);

  // Repairs Data
  const [repairs, setRepairs] = useState(() => Object.values(MOCK_REPAIRS));

  // Load live stats from backend API
  const loadStats = async () => {
    setLoadingStats(true);
    try {
      const data = await fetchAdminStats();
      setStats(data);
    } catch (err) {
      console.warn('[Admin] Live stats API failed, calculating local fallback metrics:', err);
      // Fallback metrics calculated from local state
      const inStock = products.filter((p) => p.in_stock).length;
      const pending = repairs.filter(
        (r) => ![ 'delivered', 'cancelled' ].includes((r.status || '').toLowerCase())
      ).length;
      setStats({
        total_products: products.length,
        in_stock_products: inStock,
        total_repairs: repairs.length,
        pending_repairs: pending
      });
    } finally {
      setLoadingStats(false);
    }
  };

  // Load all products from API for admin view
  const loadProducts = async () => {
    setLoadingProducts(true);
    try {
      const prods = await fetchProducts(null, null, true);

```

```

    if (Array.isArray(prods) && prods.length > 0) {
      setProducts(prods);
    }
  } catch (err) {
    console.warn('[Admin] Products fetch fallback:', err);
  } finally {
    setLoadingProducts(false);
  }
};

useEffect(() => {
  loadStats();
  loadProducts();
}, []);

// Handlers for product management connected to backend CRUD
const handleAddProduct = async (newProdData) => {
  const created = await createProduct(newProdData);
  setProducts((prev) => [created, ...prev]);
  loadStats();
  return created;
};

const handleUpdateProduct = async (updatedProdData) => {
  const updated = await updateProduct(updatedProdData.id, updatedProdData);
  setProducts((prev) =>
    prev.map((p) => (p.id === updated.id ? updated : p))
  );
  loadStats();
  return updated;
};

const handleDeleteProduct = async (productId) => {
  await deleteProduct(productId);
  setProducts((prev) => prev.filter((p) => p.id !== productId));
  loadStats();
};

// Handlers for repairs management
const handleUpdateRepairStatus = (jobSheetId, newStatus, technicianNotes) => {
  setRepairs((prev) =>
    prev.map((r) =>
      r.job_sheet_id === jobSheetId
        ? {
            ...r,
            status: newStatus,
            technician_notes: technicianNotes || r.technician_notes,
            updated_at: new Date().toISOString().replace('T', ' ').substring(0, 19)
          }
        : r
    )
  );
};

// Update stats pending repairs count
setTimeout(() => {
  setStats((prev) => {
    const pending = repairs.map((r) =>
      r.job_sheet_id === jobSheetId ? { ...r, status: newStatus } : r
    ).filter(
      (r) => ![ 'delivered', 'cancelled' ].includes((r.status || '').toLowerCase())
    ).length;
    return {
      ...prev,
      pending_repairs: pending
    };
  });
}, 50);

const handleCreateRepairJob = (newJob) => {

```

```

setRepairs((prev) => [newJob, ...prev]);
setStats((prev) => ({
  ...prev,
  total_repairs: prev.total_repairs + 1,
  pending_repairs: prev.pending_repairs + 1
}));
});

return (
  <AdminLayout
    activeTab={activeTab}
    setActiveTab={setActiveTab}
    admin={admin}
    logout={logout}
    searchQuery={searchQuery}
    setSearchQuery={setSearchQuery}
    showSearch={true}
    onRefresh={loadStats}
    loadingStats={loadingStats}
  >
    {activeTab === 'dashboard' && (
      <DashboardOverview
        admin={admin}
        stats={stats}
        loadingStats={loadingStats}
        onRefresh={loadStats}
        products={products}
        repairs={repairs}
        onNavigateTab={setActiveTab}
        onAddProduct={handleAddProduct}
        onUpdateProduct={handleUpdateProduct}
        onDeleteProduct={handleDeleteProduct}
      />
    )}

    {activeTab === 'inventory' && (
      <InventoryManager
        products={products}
        onUpdateProduct={handleUpdateProduct}
      />
    )}

    {activeTab === 'emi' && <EmiManager />}

    {activeTab === 'inquiries' && <WhatsAppInquiriesManager />}

    {activeTab === 'analytics' && <SalesAnalyticsManager />}

    {activeTab === 'products' && (
      <ProductsManager
        products={products}
        onAddProduct={handleAddProduct}
        onUpdateProduct={handleUpdateProduct}
        onDeleteProduct={handleDeleteProduct}
        searchQuery={searchQuery}
        setSearchQuery={setSearchQuery}
      />
    )}

    {activeTab === 'repairs' && (
      <RepairsManager
        repairs={repairs}
        onUpdateRepairStatus={handleUpdateRepairStatus}
        onCreateRepairJob={handleCreateRepairJob}
        searchQuery={searchQuery}
        setSearchQuery={setSearchQuery}
      />
    )}

    {activeTab === 'settings' && <SettingsManager admin={admin} />}
  </AdminLayout>
);

```

```
    </AdminLayout>
  );
}
```

frontend/src/components/Admin/ProtectedAdminRoute.jsx - Route Protection Wrapper for Authenticated Users

frontend/src/components/Admin/ProtectedAdminRoute.jsx

JavaScript/JSX

```
import React from 'react';
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../../../context/AuthContext';
import { ShieldCheck, Loader2 } from 'lucide-react';

export default function ProtectedAdminRoute({ children }) {
  const { isAuthenticated, isLoading } = useAuth();
  const location = useLocation();

  if (isLoading) {
    return (
      <div className="min-h-screen bg-slate-900 flex flex-col items-center justify-center p-4 text-white">
        <div className="flex items-center gap-3 bg-slate-800/80 px-6 py-4 rounded-2xl border border-slate-700 shadow-w-xl backdrop-blur">
          <Loader2 className="w-6 h-6 animate-spin text-primary-500" />
          <span className="text-sm font-medium tracking-wide text-slate-300">
            Verifying Admin Session...
          </span>
        </div>
      </div>
    );
  }

  if (!isAuthenticated) {
    // Redirect to admin login preserving the attempted destination
    return <Navigate to="/admin/login" state={{ from: location }} replace />;
  }

  return children;
}
```

frontend/src/components/Admin/Dashboard/DashboardDataGrid.jsx

JavaScript/JSX

```
import React from 'react';
import RecentProducts from './RecentProducts';
import RecentRepairs from './RecentRepairs';

export default function DashboardDataGrid({
  products = [],
  repairs = [],
  onNavigateTab
}) {
  return (
    <div className="grid grid-cols-1 lg:grid-cols-2 gap-4 sm:gap-5">
      <RecentProducts
        products={products}
        onNavigateTab={onNavigateTab}
      />

      <RecentRepairs
        repairs={repairs}
        onNavigateTab={onNavigateTab}
      />
    </div>
  );
}
```

7. Installation, Execution & Operations Guide

7.1 Running the Backend

```
# 1. Install Python dependencies
pip install -r backend/requirements.txt

# 2. (Optional) Initialize / Seed Admin Account
python backend/seed_admin.py --username admin --password your_password

# 3. Start the FastAPI ASGI server
python backend/run.py
```

The backend will listen on `http://localhost:8000`. Test with:

- Health check: `http://localhost:8000/api/health`
- Swagger documentation: `http://localhost:8000/docs`

7.2 Running the Frontend

```
# 1. Install Node.js packages
cd frontend
npm install

# 2. Start Vite development server
npm run dev
```

The frontend will be accessible at `http://localhost:5173`.

System Status: Verified & Operational

Both the backend API and frontend SPA are verified to run simultaneously with real-time proxying, zero database configuration required (SQLite engine runs out of the box), and complete bilingual Hindi/English capabilities.